6/6

Security monitoring assisté par IA destiné aux PMEs

Romain Christen

For the completion of the degree Master universitaire

ès Sciences en science forensique

Faculty of Law, Criminal Justice and Public Administration

School of Criminal Science, University of Lausanne

In partnership with SeLabs.ai and SecuLabs SA

Under the direction of Prof. Thomas Souvignet at the University of Lausanne

Under the supervision of Dr. Lionel Beaugé and Dr. Giao Nguyen at SeLabs.ai

Mr. Samuel Bovy and Ms. Emilie Kuhn at SecuLabs SA

UNIL | Université de Lausanne

Ai Swiss Experts Labs

SecuLabs IT SECURITY SERVICES

Abstract

As cyber threats become more prevalent and sophisticated, this master's thesis aims to enable the design of a high-performance, energy-efficient anomaly detection solution based on artificial intelligence, specifically tailored to the environments of SecuLabs and SeLabs.

To this end, a design framework for the future SOC-AI solution was defined, allowing for anticipation of the nature of available data, the technical architecture to be used, and the envisaged processing pipeline. A state-of-the-art review of existing AI solutions and relevant datasets was conducted, and two open-source solutions, LogLLM and LogAI, were selected for a benchmarking phase, taking into account both detection performance and energy consumption. The results demonstrate that LogLLM provides the best detection results, though at the cost of higher energy consumption and processing time. The Deep Learning models integrated into LogAI are faster and more efficient while remaining effective. The top performing models based on F1-score, in descending order, are : supervised LSTM, supervised CNN, supervised Transformer, and unsupervised LSTM. The Machine Learning models proposed by LogAI turned out to be ineffective in this context. Various improvement directions for LogLLM were examined, in particular adjusting the number of epochs during LogLLM training, substituting the main LLM of the solution, and contextualizing the generated alerts via an additional LLM.

Keywords : Anomaly Detection, Intrusion Detection, Artificial Intelligence, Deep Learning, Large Language Model, Security Operations Center, Event Logs, Benchmark, Cybersecurity Datasets, Security Monitoring

Résumé

Dans un contexte de multiplication et de sophistication des menaces cybernétiques, ce mémoire de master vise à permettre la conception d'une solution de détection d'anomalies basée sur l'intelligence artificielle (IA), performante et économe en énergie, spécifiquement adaptée à l'environnement des sociétés SecuLabs et SeLabs.

Pour cela, un cadre de conception de la future solution SOC-IA a été défini, permettant d'anticiper la nature des données disponibles, l'architecture technique utilisée et la chaîne de traitement envisagée. Un état de l'art des solutions d'IA existantes et des jeux de données pertinents a été réalisé, deux solutions libres d'accès, LogLLM et LogAI, ont été sélectionnées pour une phase de benchmark, tenant compte à la fois des performances de détection et de la consommation énergétique. Les résultats démontrent que LogLLM offre les meilleurs résultats en termes de détection, mais au prix d'un coût énergétique et temps de traitement élevé. Les modèles de Deep Learning intégrés dans LogAI sont plus rapides et économes et restent performants. Les plus performants selon le F1-score, par ordre décroissant, sont : LSTM supervisé, CNN supervisé, Transformer supervisé, LSTM non supervisé. Les modèles de Machine Learning proposés par LogAI se sont eux révélés inefficaces dans ce contexte. Plusieurs pistes d'amélioration de LogLLM ont été examinées, en particulier l'ajustement du nombre d'epochs lors de l'entraînement de LogLLM, la substitution du LLM principal de la solution et la contextualisation des alertes générées via un LLM supplémentaire.

Mots-clés : Détection d'anomalies, Détection d'intrusions, Intelligence artificielle, Apprentissage profond, Modèle de langage de grande taille, Centre d'opérations de sécurité, Journaux d'événements, Évaluation comparative, Jeux de données de cybersécurité, Surveillance de la sécurité

Utilisation de l'intelligence artificielle

Dans le cadre de ce travail, différents outils d'intelligence artificielle ont été utilisés conformément aux directives de l'École des sciences criminelles (ESC) de l'Université de Lausanne, telles que décrites dans le document « Cadre d'usage de l'intelligence artificielle (IA) à l'ESC (Version 2025) ». Les outils suivants ont été utilisés :

- ChatGPT, pour l'amélioration du style rédactionnel et l'aide à la traduction ;
- ChatGPT, Gemini et Claude, pour l'aide à la programmation.

Tous ces outils ont été utilisés via l'interface Mammouth.ai (URL : https ://mammouth.ai). L'auteur confirme que la version finale du texte reflète fidèlement son travail et ses idées originales et qu'il en assume l'entière responsabilité.

Remerciements

Je tiens à exprimer ma gratitude à M. Dominique Vidal, CEO de SecuLabs SA et SeLabs.ai, pour m'avoir offert l'opportunité d'intégrer son entreprise et de réaliser ce mémoire de master. Je remercie le professeur Thomas Souvignet de l'Université de Lausanne pour sa supervision. J'adresse une mention particulière au Dr. Lionel Beaugé (SeLabs.ai) pour son accompagnement attentif et ses conseils précieux. Je remercie également Mme Emilie Kuhn et M. Samuel Bovy (SecuLabs SA) ainsi que le Dr. Giao Nguyen (SeLabs.ai) pour leur accompagnement et leur soutien tout au long de ce travail. Je souhaite aussi remercier toute l'équipe de SecuLabs SA et SeLabs.ai pour leur accueil chaleureux. Enfin, un merci à M. Christoph Landolt pour ses recommandations généreuses.

Abréviations

AD	Anomaly Detection
CIC	Canadian Institute for Cybersecurity
CNN	Convolutional Neural Network
CSE	Communications Security Establishment
DDoS	Distributed Denial of Service
DL	Deep Learning
DoS	Denial of Service
EDR	Endpoint Detection and Response
GAN	Generative Adversarial Network
GNN	Graph Neural Network
GPO	Group Policy Object
GRU	Gated Recurrent Unit
IA	Intelligence Artificielle
ID	Intrusion Detection
IDS	Intrusion Detection System
IOC	Indicator Of Compromise
IP	Internet Protocol
LSTM	Long Short-Term Memory
LLM	Large Language Model
LOF	Local Outlier Factor
ML	Machine Learning
PCAP	Packet CAPture
PMEs	Petites et Moyennes Entreprises
RNN	Recurrent Neural Network
SSH	Secure SHell
SOC	Security Operation Center
SQL	Structured Query Language
SSL	Secure Sockets Layer
VLAN	Virtual Local Area Network

Table des matières

1 Introduction **1**

2 Revue de littérature **2**

2.1 Anomalies en cybersécurité 2

2.1.1 Typologie des anomalies en cybersécurité 2

2.1.2 Signification des anomalies en cybersécurité 2

2.1.3 Sources et typologies de traces pour la détection d'anomalies 4

2.1.4 Méthodes traditionnelles de détection d'anomalies et d'intrusions . . . 5

2.2 Modèles d'IA pour la cybersécurité 6

2.2.1 Notions fondamentales et types d'entraînements 6

2.2.2 Avantages et limites de l'IA en détection d'anomalies et d'intrusions . . 8

2.2.3 État de l'art des modèles d'IA pour la détection d'anomalies et d'intrusions 9

2.3 État de l'art des jeux de données pour l'évaluation de modèles d'IA en détection d'anomalies et intrusions 12

3 Cadre de conception d'une solution SOC-IA adaptée à l'environnement de SecuLabs & SeLabs **15**

3.1 Origine du Projet et Perspectives sur les Sources de Données Futures 15

3.2 Futur Fonctionnement de la solution 15

3.3 Infrastructure de développement 16

3.4 Future Chaîne de traitement des données 17

3.4.1 Sélection des traces pertinentes 17

3.4.2 Collecte des données 18

3.4.3 Annotation et profil d'activité normale 18

4 Méthodologie **19**

4.1 Sélection des solutions IA 19

4.1.1 Description de LogLLM et adaptations apportées 19

4.1.2 Description de LogAI et adaptations apportées 20

4.2 Sélections des jeux de données 21

4.3 Benchmarking des solutions 21
4.3.1 Surveillance de la consommation de ressources 22
4.3.2 Description et Prétraitement des jeux de données pour LogLLM 22
4.4 Tentatives d'amélioration de LogLLM 29
4.4.1 Augmentation du nombre d'epochs 29
4.4.2 Remplacement de Meta-Llama-3-8B 30
4.4.3 Interprétation des réponses positives de LogLLM 30

5 Résultats **31**
5.1 Benchmarking des solutions 31
5.2 Tentatives d'amélioration de LogLLM 32
5.2.1 Augmentation du nombre d'epochs 32
5.2.2 Remplacement de Meta-Llama-3-8B 32
5.2.3 Interprétation des réponses positives de LogLLM 33

6 Discussion **34**
6.1 Benchmarking des solutions 34
6.1.1 Comparaison de LogLLM et des algorithmes de LogAI 34
6.1.2 Évaluation de LogLLM sur différents jeux de données 35
6.2 Tentatives d'amélioration de LogLLM 36
6.2.1 Augmentation du nombre d'epochs 36
6.2.2 Remplacement de Meta-Llama-3-8B 36
6.2.3 Interprétation des réponses positives de LogLLM 37
6.3 Cas d'utilisation du modèle : Forensique vs SOC 37
6.4 Propositions de développements futurs 38
6.4.1 Évolutions possibles : modèles spécialisés et approches d'ensemble . . . 38
6.4.2 Fonctionnalités supplémentaires 38

7 Conclusion **39**

1 Introduction

Dans un contexte où la complexité et la fréquence des menaces cybernétiques ne cessent d'augmenter, la capacité à détecter rapidement les anomalies et intrusions constitue un enjeu majeur pour toutes les organisations. Les petites et moyennes entreprises (PMEs), en particulier, demeurent vulnérables : leurs ressources limitées, tant sur le plan humain que financier, ainsi qu'une expertise technique parfois insuffisante, rendent difficile la gestion de menaces toujours plus sophistiquées. Par ailleurs, les approches traditionnelles de détection, fondées sur des règles ou des signatures d'attaques connues, montrent leurs limites face à l'évolution constante des tactiques adverses. Face à ce constat, l'intelligence artificielle (IA) offre de nouvelles perspectives pour automatiser et renforcer les capacités de surveillance en cybersécurité. C'est dans cet esprit que les entreprises SecuLabs et SeLabs, expertes en sécurité informatique et en IA, souhaitent explorer le développement d'une solution SOC-IA (Security Operations Center assisté par IA) adaptée à leur propre environnement et aux besoins des PMEs.

La question de recherche centrale de ce mémoire s'énonce ainsi : **Comment concevoir une solution de surveillance de sécurité basée sur l'IA pour la détection d'anomalies, performante et économe en énergie, adaptée à l'environnement de SecuLabs & SeLabs ?**

Pour y répondre, ce mémoire s'appuie d'abord sur une revue de la littérature portant sur la détection d'anomalies (Anomaly Detection, AD) en cybersécurité, les solutions d'IA pertinentes et les jeux de données pour l'évaluation de telles solutions. Il propose ensuite un cadre de conception d'une solution SOC-IA commercialisable par SecuLabs et SeLabs comprenant : les sources de données disponibles, l'architecture technique et la chaîne de traitement envisagée. S'en suit une méthodologie incluant la sélection, l'adaptation et le benchmark de solutions IA (LogLLM et LogAI) à partir de jeux de données d'évaluation, ainsi que des tentatives d'amélioration de LogLLM. Finalement, les performances et la consommation des modèles sont analysées, et des pistes d'amélioration et de développements futurs sont discutées.

2 Revue de littérature

2.1 Anomalies en cybersécurité

2.1.1 Typologie des anomalies en cybersécurité

Dans le contexte de la cybersécurité, les anomalies sont définies comme des observations qui s'écartent significativement des schémas normaux ou des comportements attendus dans un jeu de données. La typologie proposée par Foorthuis (2018) identifie six types d'anomalies :

1. Anomalies de valeur extrême : valeurs numériques inhabituellement élevées, basses ou rares par rapport à la distribution générale des données.

   Exemple : taille anormalement élevée d'un paquet réseau.

2. Anomalies de classe rare : observations appartenant à une classe ou catégorie très peu représentée dans le jeu de données.

   Exemple : utilisation d'un port jamais utilisé auparavant.

3. Anomalies multidimensionnelles numériques : associations inhabituelles de plusieurs variables continues, qui, prises séparément, sont normales mais qui, ensemble, révèlent une anomalie.

   Exemple : envoi régulier d'un grand nombre de paquets de taille élevée.

4. Anomalies de classe rare multidimensionnelles : combinaisons inhabituelles de plusieurs variables catégorielles.

   Exemple : succession d'identifiants d'événement caractéristiques d'une attaque spécifique.

5. Anomalies mixtes multidimensionnelles : anomalies résultant d'une combinaison atypique de valeurs continues et catégorielles.

6. Anomalies mixtes simples : observations présentant à la fois une valeur continue extrême et une catégorie rare.

2.1.2 Signification des anomalies en cybersécurité

En cybersécurité, les anomalies peuvent indiquer des comportements malicieux, mais ce n'est pas systématiquement le cas. Elles peuvent également résulter d'une activité inhabituelle, sans qu'elle soit nécessairement malveillante. Par exemple, Highnam et al. (2021) ont choisi,

lors de la création de leur jeu de données BETH, de classifier les anomalies en deux catégories distinctes : *Evil* pour une observation malicieuse, et *Suspicious* pour une activité inhabituelle ou pour une valeur aberrante sans lien avéré avec une activité malveillante.
De plus, une anomalie peut provenir d'une défaillance système ou d'une erreur critique n'ayant aucun rapport direct avec une attaque. C'est notamment le cas pour les anomalies identifiées dans des jeux de données tels que HDFS (Xu et al., 2009) ou BGL (Adam Oliner and Jon Stearley, 2007).
Les anomalies révélatrices d'attaques ou d'intrusions (lorsque l'attaquant entre sur le système) sont particulièrement pertinentes pour ce travail. Elles se manifestent lorsque des techniques d'attaque, comme celles décrites dans le cadre MITRE ATT&CK (Adversarial Tactics, Techniques & Common Knowledge) (MITRE, 2025), sont employées par un adversaire.
Le MITRE ATT&CK est une base de connaissances publique qui recense et catégorise les méthodes employées par les cyberadversaires dans des environnements réels. Sa taxonomie hiérarchique s'articule autour de trois niveaux : les *tactiques*, correspondant aux objectifs stratégiques poursuivis lors des différentes phases d'une attaque ; les *techniques*, qui désignent les méthodes spécifiques employées pour atteindre chaque tactique ; les *procédures*, qui illustrent la mise en œuvre concrète de ces techniques dans des cas d'attaques observés. Cette classification facilite l'identification des comportements malveillants au cours d'incidents de sécurité.

La détection d'une anomalie causée par un incident de sécurité permet de révéler cet incident et d'initier un processus de réponse à incident. Patrick Kral (2011) décompose ce processus en six phases : la préparation, l'identification, le confinement, l'éradication, la récupération et le retour d'expérience. La mise en place d'une solution de détection d'anomalie relève de la première phase, la préparation. Cette étape vise à mettre en place des outils pour mieux prévenir et détecter les incidents potentiels. Lorsqu'une anomalie est effectivement détectée, on entre dans la deuxième phase, l'identification, dont l'objectif est de détecter une attaque menée sur le système, puis de déterminer sa nature et son étendue.

Ainsi, la capacité à détecter les anomalies est essentielle pour garantir une réponse à inci-

dent et limiter l'impact d'un incident de sécurité.

2.1.3 Sources et typologies de traces pour la détection d'anomalies

L'étape d'identification repose sur la collecte d'informations issues de multiples sources, telles que les captures réseau, les messages d'alerte provenant d'outils de sécurité (comme les systèmes de détection d'intrusion [IDS], les solutions de détection et de réponse pour les terminaux [EDR], les pare-feux), ainsi que les journaux d'événements (Patrick Kral, 2011). D'autres outils récents d'analyse comportementale des utilisateurs et entités constituent également une source de traces intéressante (Nexthink, 2025; Fortinet, 2025).

Journaux d'événements Un journal d'événements (ou Event Log en anglais) est un enregistrement des événements survenant au sein des systèmes et réseaux d'une organisation. Les logs se composent d'entrées qui fournissent des informations relatives à la majorité des actions réalisables sur le système, ce qui permet de retracer l'activité du système et de faciliter les analyses de sécurité (María del Carmen Prudente Tixteco et al., 2016; K Kent and M P Souppaya, 2006). Microsoft définit un événement de la manière suivante « toute occurrence significative dans le système ou dans un programme qui nécessite une notification à l'utilisateur ou une inscription dans un journal d'événement » (Kinza Yasar and Alexander S. Gillis, 2023).

Une entrée de log typique contient plusieurs informations clés telles que la date et l'heure de l'événement, l'utilisateur concerné, le nom de la machine, l'identifiant d'événement, la source ayant généré l'événement, ainsi que le type d'événement et le niveau de sévérité (Kinza Yasar and Alexander S. Gillis, 2023).

Il existe différents types de journaux d'événements selon les systèmes : sous Windows, on retrouve les Windows Event Logs (Wineventlog) et, si implémenté, les journaux enrichis générés par Sysmon (System Monitor), offrant un niveau de détail supplémentaire (Mark Russinovich and Thomas Garnier, 2024) ; sous Linux, des journaux standards comme syslog ou auth.log, ce dernier étant dédié à la journalisation des authentifications.

Les Windows Event Logs comportent un champ appelé Level qui renseigne sur la sévérité de l'événement. Ce champ permet d'estimer le niveau de risque associé selon plusieurs catégories : Information, Critical, Warning, et Error (Kinza Yasar and Alexander S. Gillis, 2023).

Ces journaux sont enregistrés au format *.evtx* et répartis dans plusieurs fichiers selon leur catégorie : *Application.evtx* enregistre les événements liés aux applications et logiciels installés ; *System.evtx* regroupe les événements générés par le système d'exploitation, les pilotes et les services système ; *Security.evtx*, le plus crucial pour la sécurité, contient les événements relatifs à la sécurité, tels que les tentatives de connexion, les accès à des ressources protégées ainsi que tous les événements d'audit avancés (Kinza Yasar and Alexander S. Gillis, 2023).
Pour renforcer leur posture de sécurité, les organisations peuvent gérer leurs GPO (Group Policy Object) afin d'auditer de nouveaux événements ou d'enregistrer des informations supplémentaires pour certains types d'activités : c'est ce que l'on appelle les événements d'audit avancés (Microsoft, 2018).

Captures réseau Les captures réseau représentent une autre source essentielle d'informations dans le cadre de l'identification des incidents de sécurité. Elles consistent à enregistrer le trafic réseau, c'est-à-dire l'ensemble des paquets échangés sur un réseau pendant une période donnée. Ces enregistrements sont généralement réalisés au format PCAP (Packet CAPture), lequel conserve une copie brute, paquet par paquet, du trafic observé.
Une capture réseau offre des informations contenues dans l'en-tête de chaque paquet : l'horodatage du paquet ; la taille du paquet, aussi bien la taille capturée que celle effectivement transmise sur le réseau, fournissant des renseignements sur le volume des données échangées. Des informations peuvent également être tirées des données brutes du paquet, telles que : les adresses source et destination, au niveau MAC (Media Access Control) et au niveau IP (Internet Protocol), permettant d'identifier les dispositifs communiquant ; les ports source et destination ; le protocole utilisé pour chaque conversation (Richard Gargan, 2024).

2.1.4 Méthodes traditionnelles de détection d'anomalies et d'intrusions

L'analyse statistique (non IA) est une méthode permettant de détecter des anomalies dans les données de sécurité informatique. Son principe fondamental consiste à établir un profil décrivant une activité normale. Ce profil, construit à partir de données historiques et de mesures statistiques, définit ce qu'est un comportement normal sur le système. Une fois ce profil de référence établi, il devient possible de détecter les données qui s'en écartent statistiquement de manière significative, et qui correspondent donc à des comportements anormaux. Ces

données sont alors classifiées comme des anomalies. Pour cette tâche, différentes techniques statistiques sont couramment utilisées, telles que le calcul du Z-score, l'écart interquartile, le test de Grubbs, ou encore des méthodes reposant sur la médiane (Domański, 2020).

La détection d'anomalies peut aussi s'appuyer sur la visualisation des données. Dans ce contexte, la visualisation permet de faire ressortir des anomalies qui peuvent ensuite être identifiées par l'expertise de l'opérateur. Des outils comme OpenSearch Dashboards ou Kibana proposent, via des tableaux de bord dynamiques, des représentations facilitant l'identification visuelle des anomalies (Opensearch, 2025b; Elasticsearch, 2025).

Par ailleurs, la recherche d'intrusions s'effectue fréquemment en s'appuyant sur des indicateurs de compromission (Indicators Of Compromise, IOCs), tels que les signatures de virus, des adresses IP ou des domaines suspects. Elle peut également être basée sur les journaux d'événements afin d'identifier des comportements anormaux sur la base d'événements ou corrélation de plusieurs événements (María del Carmen Prudente Tixteco et al., 2016).

Dans ce cadre, la détection par règles ou signatures est une méthode largement employée. Un ensemble de règles logiques ou de signatures prédéfinies est utilisé pour signaler tout événement connu pour être anormal. Ces règles s'appuient sur une connaissance préalable des menaces, elles peuvent se déclencher sur la base d'un IOC. *Exemple : tentative de connexion depuis une IP sur liste noire.* Ou selon des patterns d'activité prédéfinis. *Exemple : trop grand nombre d'échecs de connexion en peu de temps.*

De nombreux IDS implémentent ce type de règles. Elles offrent une grande lisibilité et interprétabilité des alertes (Milvus, 2025).

2.2 Modèles d'IA pour la cybersécurité

2.2.1 Notions fondamentales et types d'entraînements

« L'intelligence artificielle désigne l'ensemble des techniques permettant à une machine de simuler certains processus cognitifs humains, tels que l'apprentissage, le raisonnement ou la perception. » (Alzubaidi et al., 2021). Le machine learning (ML ou apprentissage machine) est une sous-catégorie de l'IA qui montre une abilité d'apprentissage sans être explicitement programmé pour, en basant son apprentissage sur une grande quantité de données (Joshi, 2020). Le deep learning (DL ou apprentissage profond) est une sous-catégorie du machine

learning qui utilise plusieurs couches de réseaux de neurones profonds. Cela permet de capter des motifs complexes dans les données (LeCun et al., 2015). La figure 1 illustre cette relation d'inclusion : le DL est un sous-ensemble du ML, lui-même inclus dans le champ plus large de l'IA.

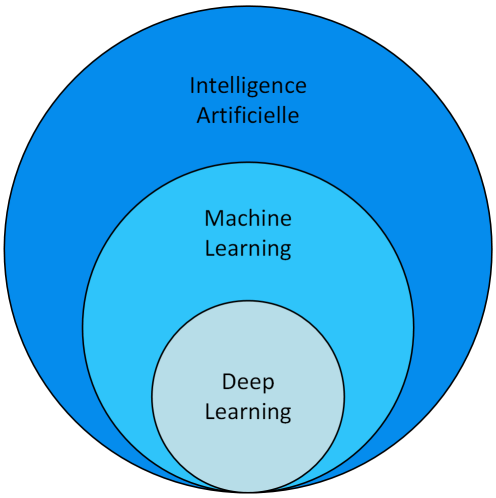

FIGURE 1 – L'IA et ses sous-ensembles : machine learning et deep learning

Une caractéristique essentielle des modèles est le type de supervision.

Entraînement supervisé : ces modèles sont entraînés sur un jeu de données annotées. C'est-à-dire que les données sont explicitement catégorisées (Joshi, 2020). Pour la détection d'anomalies en cybersécurité on parle généralement de classification binaire, c'est-à-dire que les échantillons sont annotés selon deux catégories : activité normale ou anomalie (Siam et al., 2025). En détection d'intrusions (Intrusion Detection, ID) il est courant d'observer une classification multi-classe, c'est-à-dire avec un plus grand nombre de catégories. Elles comprennent souvent l'activité normale et plusieurs types d'attaques (Canadian Institute for Cybersecurity, 2015).

Entraînement non supervisé : ces modèles sont entraînés sur un jeu de données non annotées. L'annotation de jeux de données de grandes tailles est laborieuse, cette méthode permet d'éviter cette étape. En cybersécurité, elle est également intéressante pour sa capacité de détection de menaces encore inconnues (Siam et al., 2025).

Entraînement semi-supervisé : ici les modèles sont entraînés à la fois sur des données annotées et non annotées. Cette méthode permet une annotation incomplète et regroupe à la fois les avantages de détection de menaces inconnues et la reconnaissance de motifs d'attaques appris

durant l'entraînement supervisé (Siam et al., 2025).

Supervision one-class : c'est le cas lorsque toutes les données appartiennent à une seule classe. En cybersécurité il est courant d'utiliser uniquement des échantillons d'activité normale afin d'établir un profil d'activité normale (Wang et al., 2020).

Reinforcement learning : ici le jeu d'entraînement n'est pas annoté mais le modèle apprend en recevant des récompenses ou punitions selon ses actions (Joshi, 2020).

2.2.2 Avantages et limites de l'IA en détection d'anomalies et d'intrusions

L'utilisation de l'IA en détection d'anomalies et d'intrusions présente de nombreux avantages. Les modèles sont capables d'être entraînés sur de grandes quantités de données historiques, ce qui leur permet de détecter des menaces complexes et évolutives. Contrairement aux systèmes traditionnels, basés sur des règles prédéfinies, les modèles d'IA peuvent identifier des menaces encore inconnues, en particulier lorsqu'ils intègrent une étape d'entraînement non supervisé (Siam et al., 2025). Un autre avantage considérable, notamment pour certains modèles de DL, est leur capacité à analyser d'importantes quantités de données non structurées (Siam et al., 2025).

Un dernier atout des modèles d'IA réside dans leur flexibilité et évolutivité. En effet, contrairement à une règle prédéfinie, un modèle peut s'adapter à un nouvel environnement. Toutefois, cette évolution s'accompagne d'une augmentation du risque de faux positifs, c'est-à-dire d'alertes de sécurité ne correspondant pas à de véritables incidents. Pour limiter ce phénomène, une approche consiste à effectuer un entraînement initial non supervisé, suivi d'un affinement supervisé (fine-tuning) spécifique à l'environnement cible (Guthula et al., 2025).

Parmi les principaux inconvénients, on relève la forte consommation de ressources, particulièrement prononcée pour les modèles de DL (Siam et al., 2025), et, plus encore, pour les modèles de langage de grande taille (Large Language Model, LLM). Par ailleurs, le manque d'interprétabilité de ces modèles, en particulier ceux fondés sur le DL, engendre des défis quant à la lisibilité et à l'utilisation des alertes pour des réponses à incidents efficaces (Siam et al., 2025).

2.2.3 État de l'art des modèles d'IA pour la détection d'anomalies et d'intrusions

La table 1 présente les différentes solutions IA, dont le code source est en libre accès, et qui sont utilisables dans le cadre de la détection d'anomalies et d'intrusions.

Apprentissage profond (DL) Parmi les différents types de modèle de DL, on retrouve les transformers. Il s'agit d'une architecture de réseaux de neurones fondée sur l'attention multi-tête. L'attention, plus précisément la self-attention, est un mécanisme qui analyse les relations entre chaque token (une unité de texte comme un mot ou une partie de mot) et tous les autres tokens de la séquence. Elle permet au modèle de déterminer l'importance relative de chaque élément de l'entrée, quelle que soit sa position, ce qui améliore la prise en compte du contexte global. Le recours à l'attention multi-tête signifie que plusieurs mécanismes d'attention (appelés têtes) sont utilisés en parallèle, afin de capturer différentes relations au sein de la séquence (Vaswani et al., 2023; Kalra, 2024). Les transformers sont notamment utilisés en traitement automatique du langage naturel pour comprendre, interpréter et générer du texte en langage humain (Khurana et al., 2023). Dans ce contexte, les LLMs représentent une avancée majeure, reposant sur l'architecture des transformers et entraînés sur de très grands corpus textuels.
Certaines solutions de détection d'anomalies ou intrusions utilisent l'architecture transformers sous sa forme non LLM. C'est le cas de Neurallog et logAI (Le and Zhang, 2021; Cheng et al., 2023). D'autres solutions utilisent des LLMs. Dans ce cas les solutions reprennent souvent des LLMs libre d'accès pré-entraînés par des grands groupes et y appliquent un second entraînement (fine-tuning) afin de l'adapter à la tâche spécifique de détection. C'est le cas de LogGPT qui utilise le modèle pré entraîné GPT-2 d'OpenAI (Han et al., 2023) ou de LogLLM qui utilise les modèles Bert de Google et Llama de Meta (Guan et al., 2025). On peut citer des solutions supplémentaires utilisant l'architecture transformers comme CAT, Log Bert, Flow-Transformer (Zhang et al., 2022; Guo et al., 2021; Manocchio et al., 2024) ou RAPID qui a la particularité de ne nécessiter aucun entraînement (No et al., 2023).

Les Convolutional Neural Networks (CNN) ont la particularité d'avoir une couche de convolution qui permet d'extraire des caractéristiques et de les représenter vectoriellement. Ces

Solution	**date de sortie**	**supervision**	**architecture**	**spécificité de détection**
LogLLM (Guan et al., 2025)	2025	Supervisé binaire	BERT, LLama	Traitement de logs bruts, AD
CNN-LSTM (Bamber et al., 2025)	2025	Supervisé binaire	CNN, LSTM	ID
FastLogAD (Lin et al., 2024)	2024	One class	GAN, BERT	Logs AD
LogSD (Xie et al., 2024)	2024	One class	CNN	Logs systèmes AD
OneLog (Shayan Hashemi and Mika Mäntylä, 2024)	2024	Supervisé binaire	CNN	Logs logiciels AD
Enhanced CNN (Sharma et al., 2024)	2024	Supervisé multi-classe	CNN	ID réseau temps réel
XG-NID (Farrukh et al., 2025)	2024	Supervisé multi-classe	GNN, LLM	ID réseau et interprétation via LLM
LogAI (Cheng et al., 2023)	2023	À choix	À choix	Traitement de logs bruts, AD et benchmark
LogGPT (Han et al., 2023)	2023	One class puis reinforcement learning	GPT	Logs AD
Logs2graph (Li et al., 2024)	2023	One class	GNN	Logs AD
GLAD (Yao et al., 2025)	2023	One class	GNN	Logs AD
RAPID (No et al., 2023)	2023	Non supervisé	LLM	Logs AD sans entraînement
FlowTransformer (Manocchio et al., 2024)	2023	Supervisé binaire	Transformer à choix	ID réseau
NetFound (Guthula et al., 2025)	2023	Non supervisé puis finetuning supervisé	Transformer	ID adaptable
Deep-Loglizer (Chen et al., 2022)	2022	À choix	DL à choix	Traitement de logs bruts, AD
LogGD (Xie et al., 2022)	2022	Supervisé binaire	GNN, GNN transformer	Logs AD
CAT (Zhang et al., 2022)	2022	Non supervisé	Transformer	Logs AD
Log Bert (Guo et al., 2021)	2021	Non supervisé	BERT	Logs AD
PLELog (Yang et al., 2021)	2021	Non supervisé	GRU	Logs AD
NeuralLog (Le and Zhang, 2021)	2021	Supervisé binaire	Transformer	Logs AD
CNN Log AD (Smith, 2025)	2021	Supervisé binaire	CNN	Traitement de logs bruts, AD
OC4Seq (Wang et al., 2020)	2020	One class	RNN	Logs AD
DeepLog (Du et al., 2017)	2017	One class	LSTM	Logs systèmes AD
Loglizer (He et al., 2016)	2016	À choix	ML à choix	Traitement de logs bruts, AD
Opensearch AD (Amit Galitzky, 2025)	-	Non supervisé	ML	Logs et trafic réseau AD

Table 1 – Solutions IA pour la détection d'anomalies et d'intrusions.

modèles sont largement utilisés pour la classification d'images (NEXA, 2025). Ils se retrouvent également dans des solutions de détection d'anomalies ou d'intrusions comme Enhanced CNN, LogSD, OneLog ou CNN Log AD (Sharma et al., 2024; Xie et al., 2024; Shayan Hashemi and Mika Mäntylä, 2024; Smith, 2025).

Réseaux de Neurones Graphiques (Graph Neural Networks, GNN) : Un graphe est une structure composée d'un ensemble de nœuds reliés entre eux par des arêtes. Les nœuds représentent des entités, tandis que les arêtes modélisent les relations entre ces entités. Les GNN encodent les informations relatives aux nœuds ainsi qu'à leur structure relationnelle sous forme de vecteurs caractéristiques (embeddings), ce qui permet d'obtenir une représentation numérique du graphe (Kassel, 2023). Des solutions comme Logs2graph ou GLAD (Li et al., 2024; Yao et al., 2025) utilisent des modèles GNN pour la détection d'anomalies dans les logs.

La famille des réseaux de neurones récurrents (Recurrent Neural Network, RNN) regroupe des architectures conçues pour traiter des données séquentielles, en tenant compte de l'historique des événements (Afshine Amidi and Shervine Amidi, 2025). Pour l'analyse de logs, cette approche permet d'examiner une succession d'événements qui pourrait, par exemple, révéler une attaque. Deux variantes couramment utilisées pour la détection d'anomalies sont le Long Short-Term Memory (LSTM) et le Gated Recurrent Unit (GRU), qui gèrent efficacement les dépendances temporelles dans les données séquentielles (Afshine Amidi and Shervine Amidi, 2025). Par exemple, DeepLog repose sur un modèle LSTM (Du et al., 2017), tandis que PLE-Log utilise un GRU (Yang et al., 2021).

Une solution, FastLogAD (Lin et al., 2024), utilise dans sa chaîne de traitement un réseau antagoniste génératif (Generative Adversarial Network, GAN). Une telle architecture est composée de deux réseaux de neurones : un générateur et un discriminateur qui s'affrontent et s'améliorent grâce à cette compétition (Google, 2025).

Machine Learning Certaines solutions comme LogAI, Loglizer ou Opensearch AD (Cheng et al., 2023; He et al., 2016; Amit Galitzky, 2025) permettent l'utilisation de ML, en particulier des méthodes spécialisées dans la détection d'anomalies.

Le One-Class Support Vector Machine (One-Class SVM) est un modèle de ML conçu pour la détection d'anomalies. Il définit une frontière autour des données normales et identifie les points qui s'en écartent. Il est entraîné avec supervision one-class en utilisant uniquement la classe normale. Ses caractéristiques le rendent particulièrement adapté à la détection d'anomalies en cybersécurité (Zhang et al., 2015).

L'Isolation Forest est un algorithme d'apprentissage non supervisé destiné à la détection d'anomalies. Il repère les points atypiques en les isolant via des partitions aléatoires dans une forêt, constituée de multiple arbres de décision (Conor O'Sullivan, 2024).

Le Local Outlier Factor (LOF) est un algorithme non supervisé de détection d'anomalies qui identifie les points atypiques en comparant leur densité (c'est-à-dire le nombre de points présents dans leur voisinage) à celle de leurs voisins locaux, ce qui permet de repérer des anomalies relatives à des regroupements locaux dans les données (Scikit-learn, 2025).

Solutions mixtes Certaines solutions incluent plusieurs types d'IA. On peut citer XG-NID qui contient un GNN et un LLM (Farrukh et al., 2025) ou alors des combinaisons CNN-LSTM (Bamber et al., 2025).

2.3 État de l'art des jeux de données pour l'évaluation de modèles d'IA en détection d'anomalies et intrusions

La table 2 présente les principaux jeux de données, disponibles en libre accès, utilisables dans le cadre de l'évaluation de modèles d'IA en détection d'anomalies et d'intrusions.
La littérature distingue deux grandes catégories de modèles : les modèles de détection d'anomalies et ceux de détection d'intrusions.
Les modèles de détection d'anomalies sont généralement entraînés sur des jeux de données issus de supercalculateurs, comme HDFS (Shvachko et al., 2010; Xu et al., 2009), BGL ou Thunderbird (Adam Oliner and Jon Stearley, 2007). Ces jeux, constitués de logs systèmes annotés de façon binaire (normal/anomalie), recensent surtout des anomalies liées à des incidents

Jeu de données	Date	Type d'anomalie	Type de traces	Annotation
HDFS_v1 (Xu et al., 2009)	2009	Erreurs	Logs système	binaire
HDFS_v2 (Zhu et al., 2023)	2009	Erreurs	Logs système, amélioration de l'original	non
BGL (Adam Oliner and Jon Stearley, 2007)	2007	Erreurs	Logs système	binaire
Thunderbird (Adam Oliner and Jon Stearley, 2007)	2007	Erreurs	Logs système	binaire
Openstack (Du et al., 2017; Zhu et al., 2023)	2017	Erreurs	Logs système	binaire
Hadoop (Lin et al., 2016; Zhu et al., 2023)	2016	Erreurs	Logs système	binaire
KDD'99 (Hettich, S and Bay, S. D., 1999)	1999	Intrusions	Trafic réseau	multi-classe
ISCX NSL-KDD (Tavallaee et al., 2009; Canadian Institute for Cybersecurity, 2009)	2009	Intrusions	Trafic réseau, amélioration de l'original	multi-classe
UNSW-NB15 (Moustafa and Slay, 2015)	2015	Intrusions	Trafic réseau	multi-classe
CIC-UNSW-NB15 (Hesamodin et al., 2024; Canadian Institute for Cybersecurity, 2015)	2024	Intrusions	Trafic réseau, amélioration de l'original	multi-classe
CIC-IDS-2017 (Sharafaldin et al., 2018)	2017	Intrusions	Trafic réseau	multi-classe
BCCC-CIC-IDS2017 (Shafi et al., 2025b)	2024	Intrusions	Trafic réseau, amélioration de l'original	multi-classe
CSE-CIC-IDS2018 (Canadian Institute for Cybersecurity, 2018)	2018	Intrusions	Trafic réseau, event logs Windows et Linux	multi-classe
LycoS-Unicas-IDS2018 (Cantone et al., 2024)	2018	Intrusions	Trafic réseau, amélioration de l'original	multi-classe
CSECICIDS2018 improved (Liu, Lisa et al., 2022)	2023	Intrusions	Trafic réseau, amélioration de l'original	multi-classe
BCCC-CSE-CIC-IDS2018 (Shafi et al., 2025a)	2025	Intrusions	Trafic réseau, amélioration de l'original	multi-classe
RoEduNet-SIMARGL2021 (Mihailescu et al., 2021)	2021	Intrusions	Trafic réseau	multi-classe
Edge_IIoT (Ferrag et al., 2022)	2022	Intrusions	Trafic réseau	multi-classe
ADFA-LD (Creech and Hu, 2014; Gideon Creech and Jiankun Hu, 2013; Creech, 2014)	2013	Intrusions	Logs Linux d'appels système	binaire
ADFA-WD (Creech, 2014)	2014	Intrusions	Logs Windows d'appels système	binaire
LANL Cybersecurity Dataset (Kent, 2015)	2015	Intrusions	Logs : Windows d'authentification, dns, réseau, processus Windows. Attaques redteam.	non
BETH (Highnam et al., 2021)	2021	Intrusions	Honeypots : trafic réseau, logs de processus Linux	oui
NGIDS-DS (UNSW, 2023)	2023	Intrusions	Trafic réseau, event logs Linux	binaire
Unified Host and Network Dataset (Turcotte et al., 2017)	2017	Aucune	Trafic réseau et Winevent logs	non
Mordor (Roberto Rodriguez, 2025)	2021	Techniques Mitre Attack	Pour chaque technique : Json de Winevent ou Linux logs. Activité normale. Parfois trafic réseau.	non
Atomic Red TEAM (Roberto Rodriguez and Jose Luis Rodriguez, 2022)	2022	Techniques Mitre Attack	Pour chaque technique : Json de Winevent ou Linux logs. Pas d'activité normale.	one class

Table 2 – Jeux de données libre d'accès pour l'évaluation de modèles d'IA en détection d'anomalies et intrusions.

techniques (erreurs système, pannes matérielles, problèmes réseau, échecs de processus) plutôt qu'à des attaques informatiques. Par conséquent, ces modèles sont spécialisés dans la détection d'anomalies génériques plutôt que dans celle d'attaques. Ils restent néanmoins pertinents pour la détection d'anomalies provoquées par des incidents de sécurité.

En revanche, Les modèles de détection d'intrusions sont généralement entraînés sur des jeux de données de trafic réseau annotés en multi-classe (trafic normal et plusieurs types d'attaques), tels que KDD'99, UNSW-NB15 ou CSE-CIC-IDS2018 (Hettich, S and Bay, S. D., 1999; Moustafa and Slay, 2015; Canadian Institute for Cybersecurity, 2018). Ils permettent ainsi de détecter la catégorie de l'attaque, ce qui facilite l'interprétation des résultats.

D'autres jeux de données comme ADFA-LD ou ADFA-WD sont basés sur des logs d'appels système qui sont des enregistrements des interactions bas niveau entre un programme et le système d'exploitation (Creech and Hu, 2014; Gideon Creech and Jiankun Hu, 2013; Creech, 2014).

Certains jeux de données présentent des événements spécifiques à un système d'exploitation comme les Windows Event Logs ou des journaux Linux standards, c'est le cas des projets Mordor ou Atomic Red TEAM qui fournissent, pour chaque technique du MITRE ATT&CK, un fichier json des événements engendrés (Roberto Rodriguez, 2025; Roberto Rodriguez and Jose Luis Rodriguez, 2022).

Par ailleurs, des jeux de données mixtes comme CSE-CIC-IDS2018, BETH, NGIDS-DS ou Unified Host and Network Dataset (Canadian Institute for Cybersecurity, 2018; Highnam et al., 2021; UNSW, 2023; Turcotte et al., 2017), combinent à la fois du trafic réseau et divers journaux d'événements.

On remarque toutefois un déficit de jeux de données librement accessibles contenant des messages d'alerte issus d'outils de sécurité, probablement pour des raisons de confidentialité.

L'annotation d'un jeu de données requiert soit une analyse détaillée, soit une connaissance précise du déroulement et des horodatages des incidents. Cette annotation peut reposer sur des incidents générés artificiellement, comme pour UNSW_NB15 (Moustafa and Slay, 2015), sur des scénarios d'attaque contrôlés menés par des équipes redteam, comme dans le LANL Cybersecurity Dataset (Kent, 2015), ou sur l'analyse approfondie d'attaques réelles, comme

dans le dataset BETH, collecté à partir d'un honeypot, un leurre informatique destiné à attirer les cyberattaques pour en étudier les traces (Highnam et al., 2021).

3 Cadre de conception d'une solution SOC-IA adaptée à l'environnement de SecuLabs & SeLabs

3.1 Origine du Projet et Perspectives sur les Sources de Données Futures

SecuLabs et SeLabs collaborent avec une organisation x dans le cadre d'un partenariat qui prévoit la mise en place d'un SOC et d'un puits de logs situé en Suisse dans un centre de données sécurisé. Ce puits de logs va collecter l'ensemble des données collectées par l'organisation x. Les données comprennent notamment des traces d'attaques survenues durant la guerre en Ukraine et les Jeux Olympiques 2024. Cela permettrait d'identifier des anomalies représentatives d'attaques réelles et récentes, offrant ainsi une base pertinente pour l'entraînement et l'évaluation de modèles de détection d'anomalies et intrusions, avec un potentiel de valorisation auprès de futurs clients.
Les sources de données indisponibles au moment de la réalisation de ce travail, mais dont l'intégration est prévue, sont les suivantes : pare-feux CISCO ASA, EDR SOPHOS, Windows Active Directory, pare-feux Fortinet, proxy inverse KEMP, pare-feux SOPHOS, EDR SentinelOne. Ainsi qu'un grand nombre de serveurs Windows, postes Windows et serveurs Linux. Les données disponibles seront donc mixtes. Possiblement des messages d'alerte provenant d'outils de sécurité, du trafic réseaux et des événements Windows et Linux.

3.2 Futur Fonctionnement de la solution

La solution envisagée vise à permettre l'extraction, la centralisation et l'analyse automatisée des traces issues de l'environnement d'un client afin d'y détecter des incidents de sécurité et de fournir des alertes contextualisées. Ses étapes de fonctionnement sont les suivantes :

1. *Extraction des données client :*

   Lors de son déploiement, la solution sera connectée directement au puits de logs du

client. Pour ce faire, un agent sera installé dans le puits de logs du SOC du client afin de collecter les différentes traces pertinentes. L'agent choisi est Fluent Bit, une sonde légère qui permet d'ingérer une grande quantité de données de formats et sources différents en entrée, et de les retransmettre en sortie (FluentBit, 2025).

2. *Centralisation des données client :*

   Les données seront retransmises à OpenSearch (OpenSearch, 2025), un logiciel déjà utilisé par le SOC de l'organisation x, qui permet la centralisation et l'indexation des événements. La retransmission de données se fera via une connexion TCP sécurisée sur le port 9200, conformément à la documentation officielle (FluentBit, 2025). Les données sont finalement transmises via OpenSearch py (Opensearch, 2025a) pour être traitées avec Python.

3. *Analyse des données :*

   Les données peuvent finalement être utilisées pour l'entraînement et l'inférence de modèles d'IA destinés à la détection d'anomalies et d'intrusions. Si une menace ou un événement suspect est identifié via la chaîne de traitement IA, une alerte contextualisée sera envoyée au client.

3.3 Infrastructure de développement

La figure 2 illustre l'infrastructure de développement. Afin de permettre le développement de la solution, SeLabs a mis en place un VLAN (Virtual Local Area Network) dédié, géré par une instance de virtualisation Proxmox au sein du datacenter Stack GEN01A, situé à Plan-les-Ouates, Genève, et certifié Tier 3 en termes de sécurité. L'accès à cet environnement s'effectue via une connexion VPN SSL (Secure Sockets Layer), traversant deux pare-feux et deux commutateurs (switchs) redondés.

Dans ce VLAN, deux machines virtuelles principales sont déployées : l'une héberge Opensearch pour la centralisation, l'indexation et le stockage des logs ; l'autre est dédiée à l'hébergement du modèle d'intelligence artificielle et de sa chaîne de traitement, avec une connexion SSH (Secure SHell) sécurisée vers un serveur de calcul équipé de trois cartes graphiques NVIDIA H100 NVL 94GB (NVIDIA, 2025), nécessaires à l'entraînement des modèles d'IA. Sept emplacements supplémentaires permettront d'accueillir jusqu'à dix cartes graphiques au total.

À plus long terme, l'objectif est d'échantillonner le puits de l'organisation x, actuellement utilisé en production, afin de transférer les échantillons dans un puits de stockage géré par SeLabs. Les données pourront alors être extraites par Fluent Bit puis indexées dans Opensearch pour servir à l'entraînement et à l'évaluation de modèles d'IA.

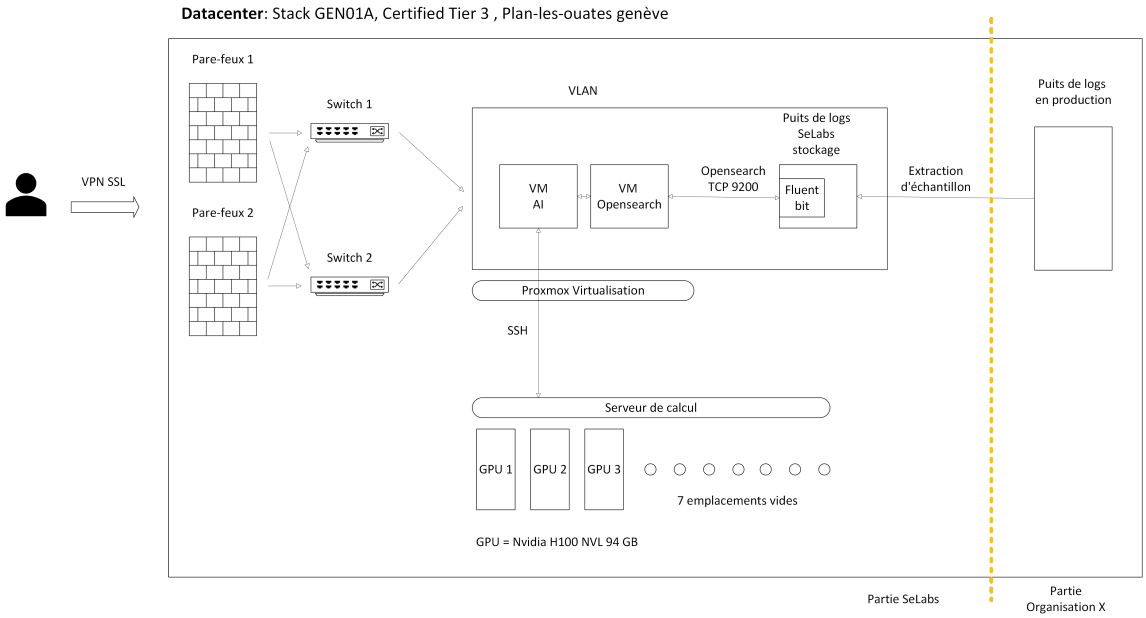

FIGURE 2 – Infrastructure de développement

3.4 Future Chaîne de traitement des données

3.4.1 Sélection des traces pertinentes

Pour le cas de l'organisation x, on peut s'attendre à des logs Windows issus des différents postes et serveurs Windows, des logs Active Directory Windows, des logs de serveur Linux mais aussi à du trafic réseau et des messages d'alertes issus des pare-feux, EDR et proxy inverses. Afin d'entraîner et utiliser efficacement un modèle, il est important de filtrer les données en sélectionnant uniquement les journaux d'événements les plus pertinents. La table 3 présente, par ordre d'importance, les différentes journaux d'événements Windows pertinents (Allen, 2022; MITRE, 2025).

Certains journaux requièrent un audit avancé via modification des GPO du client. À noter également que Sysmon, bien que pertinent, n'est pas utilisé par le SOC de l'organisation x.

Fichier	Description
Security.evtx	Evénements de sécurité et d'audit avancé
Microsoft-Windows-PowerShell/Operational.evtx	Commandes et scripts PowerShell
SMBClient/Security.evtx	Événements de sécurité relatifs au client SMB
SMBServer/Security.evtx	Événements de sécurité relatifs au serveur SMB
RemoteDesktopServices-RdpCoreTS/Operational.evtx	Événements opérationnels du noyau RDP
TerminalServices-Remote ConnectionManager/Operational.evtx	Événements sur la gestion des connexions à distance
System.evtx	Complément pour les événements système
Application.evtx	Complément pour les événements d'applications
Directory Service.evtx	Événements spécifiques aux contrôleurs de domaine Active Directory

Table 3 – journaux d'événements Windows les plus pertinents

3.4.2 Collecte des données

La collecte des données nécessitera plusieurs étapes. Il conviendra tout d'abord de rédiger un fichier de configuration et de mettre en place des analyseurs syntaxiques (parsers) Fluent Bit pour extraire toutes les sources de données ciblées. Par la suite, il sera nécessaire de conteneuriser Opensearch et de le déployer sur une machine virtuelle dédiée. Enfin, un script Python utilisant la librairie opensearch-py devra être développé afin d'extraire les données depuis OpenSearch et de les convertir au format CSV. Ce format facilitera ensuite leur exploitation sous forme de DataFrames pandas, un format adapté pour les modèles IA.

3.4.3 Annotation et profil d'activité normale

Pour entraîner un modèle supervisé ou évaluer tout modèle, quel que soit son type de supervision, il est nécessaire d'annoter tout ou une partie des données de l'organisation x, en distinguant de façon fiable les traces d'activités normales de celles résultant d'incidents de sécurité.

Dans le cadre d'un déploiement chez un client, l'établissement de son profil d'activité normale est essentiel. En effet, ce qui constitue une activité normale varie selon les clients : processus typiques, adresses IP internes, protocoles, ports utilisés, volume de trafic, durée moyenne des connexions ou plages horaires d'activité sont autant de paramètres susceptibles de différer et d'induire des faux positifs si le modèle n'est pas adapté. De plus, des facteurs tels que des politiques d'audit ou des outils de sécurité différents influencent le profil d'activité considéré comme normal pour le client.
Un premier entraînement de fond pourra être réalisé sur les données de l'organisation x, mais il sera nécessaire de procéder à une seconde phase d'entraînement afin d'ajuster le modèle au profil d'activité normale du client.

4 Méthodologie

4.1 Sélection des solutions IA

La présélection des solutions d'IA s'est appuyée sur une revue de la littérature et sur la table 1. Les solutions retenues pour un benchmark (comparaison de performances) l'ont été selon les critères suivants :

- Date de parution récente,
- Bonnes performances rapportées, à la fois par leurs auteurs et dans des études indépendantes,
- Chaîne de traitement complète (parsing, traitement des données, entraînement et inférence).

Au regard de ces critères, LogLLM (Guan et al., 2025) et LogAI (Cheng et al., 2023) ont été sélectionnées pour une analyse de leurs performances.

4.1.1 Description de LogLLM et adaptations apportées

LogLLM (Guan et al., 2025) est une solution de détection d'anomalies dans les logs reposant sur l'utilisation de LLMs. La solution peut ingérer des logs bruts et prendre en charge l'ensemble de la chaîne de traitement jusqu'à la détection d'anomalies.

La chaîne de traitement est la suivante : LogLLM ingère les logs bruts, il les parse au moyen d'expressions régulières, les convertit en séquences temporelles d'événements textuels. Ce format de séquences temporelles est couramment employé dans la détection d'anomalies au sein des logs, car il permet de capturer le contexte fourni par l'enchaînement chronologique des événements, contrairement à une approche qui ne considérerait que des événements isolés. Dans cette approche, un échantillon de données présenté au modèle n'est pas un événement unique, mais une fenêtre glissante composée de plusieurs événements consécutifs dans l'ordre chronologique.

Le fenêtrage des logs pour former les échantillons peut être réalisé selon différents critères, tels que la session réseau, une durée temporelle ou le nombre d'événements ; dans le cadre de ces expériences, le critère choisi est le nombre d'événements et la taille de cette fenêtre a été fixée à 100 événements : chaque échantillon correspond donc à une séquence de 100 événements suivant l'ordre d'apparition dans les logs.

L'étiquetage binaire des séquences temporelles suit la logique suivante :

- Une séquence est considérée comme normale si aucune anomalie n'est détectée dans la fenêtre.
- À l'inverse, la moindre anomalie présente dans la séquence entraîne le classement de l'ensemble de celle-ci comme anormale.

Pour garantir la fiabilité de l'évaluation, chaque jeu de données est divisé en deux sous-ensembles distincts : un ensemble d'entraînement (80%), et un ensemble de test (20%) strictement réservé à l'évaluation finale. Cette séparation évite tout risque de fuite d'information du jeu de test vers le modèle lors de l'apprentissage.

LogLLM utilise le modèle *bert-base-uncased* (Google, 2024), un transformer de taille modeste, couplé à un projecteur afin d'obtenir des représentations vectorielles (embeddings) des logs. Ensuite, la classification binaire (anomalie/normal) est effectuée à l'aide de *Meta-Llama-3-8B* (Meta, 2024), un LLM.

L'entraînement de LogLLM se fait de manière supervisée avec un jeu de données annotées, avec les classes binaires normal et anomalie. Pour ce faire il prend en compte des jeux de données spécifiques évalués dans l'étude de Guan et al. (2025).

Dans le cadre de ce travail, l'objectif est d'évaluer LogLLM sur une diversité de jeux de données pertinents, tous au format CSV. Pour permettre l'utilisation de jeux de données personnalisés, un nouveau script, *script_perso_window.py*, a été développé. Ce script utilise les fonctions existantes du répertoire de LogLLM et assure la conversion des fichiers CSV vers un format textuel, puis réalise le fenêtrage temporel. Il génère en sortie deux ensembles, *train.csv* et *test.csv*, respectant une répartition 80-20 entre entraînement et test. L'intégrité des sous-ensembles et l'équilibre de la répartition entre catégories normales et anormales sont vérifiés au moyen d'affichages statistiques intégrés au script Python.

Pour certains jeux de données, des adaptations spécifiques du script *script_perso_window.py* ont été nécessaires afin de répondre à des besoins particuliers. Ces versions seront détaillées dans la section consacrée au formatage des jeux de données.

4.1.2 Description de LogAI et adaptations apportées

LogAI (Cheng et al., 2023) est une solution de détection d'anomalies dans les logs. Elle intègre les fonctionnalités suivantes :

- Parsing des logs bruts avec choix de parser ;
- Fenêtrage temporel ;
- Vectorisation avec choix de méthode de vectorisation ;
- Détection d'anomalies avec choix de l'algorithme IA parmi une large gamme comprenant du ML et DL ;
- Facilitation de la comparaison des performances entre différents modèles.

Dans le cadre de cette étude, les algorithmes suivants ont été évalués avec LogAI : CNN, LSTM, architecture Transformer (non LLM), One-Class SVM, Isolation Forest et LOF. Pour ce faire deux scripts ont été développés à partir des fonctions existantes du répertoire de LogAI : *script_perso_ad_benchmarking.py* et *script_perso_ml_benchmarking.py*.

4.2 Sélections des jeux de données

L'entraînement et l'évaluation des solutions ont nécessité la sélection de jeux de données en libre accès.

Pour LogLLM, la présélection des jeux de données pertinents s'est appuyée sur la revue de littérature et la table 2. En raison de la diversité encore inconnue des données issues de l'organisation x, il a été retenu des jeux de données variés, dont certains incluent des mélanges de types de traces. Les jeux de données sélectionnés sont : HDFS_v1, CIC_UNSW_NB15, CSE_CIC_IDS2018, LANL Cybersecurity Dataset.

Pour LogAI, un seul jeu de données (HDFS_v1) a été retenu afin de limiter la complexité du benchmark, compte tenu du grand nombre d'algorithmes évalués et du fait que LogAI est directement prévu pour ce jeux de données particulier.

4.3 Benchmarking des solutions

Un benchmarking des solutions LogLLM et LogAI a été réalisé en utilisant les métriques suivantes :

- *Précision :* Proportion des échantillons prédits comme positifs qui sont effectivement positifs.

- *Rappel :* Proportion des échantillons positifs correctement identifiés comme tels par le modèle.
- *F1-score :* Moyenne harmonique entre la précision et le rappel.
- *Exactitude :* Proportion d'échantillons correctement classés parmi l'ensemble des échantillons.

Lors de la détection d'anomalies en sécurité informatique, il existe généralement un fort déséquilibre entre la classe normale (majoritaire) et celle des anomalies (minoritaire). Dans ce contexte, l'exactitude n'est pas une métrique pertinente : un modèle qui prédirait systématiquement la classe majoritaire (normal) afficherait une exactitude élevée, tout en étant incapable de détecter les anomalies.
À l'inverse, la précision, le rappel et le F1-score deviennent essentiels :

- *Précision :* Elle évalue la pertinence des alertes déclenchées, c'est-à-dire la capacité du modèle à ne pas générer un trop grand nombre de faux positifs.
- *Rappel :* Il mesure la capacité du modèle à ne pas manquer d'anomalies, c'est-à-dire à limiter les faux négatifs.
- *F1-score :* Il combine ces deux aspects.

4.3.1 Surveillance de la consommation de ressources

Un script de surveillance de l'utilisation des ressources *script_perso_monitor_resources.py* a été développé et intégré dans les codes originaux de LogLLM et LogAI afin de pouvoir surveiller la consommation de ressources de la carte graphique et du processeur lors de la vectorisation et de l'inférence des modèles sur les données parsées et fenêtrées. Une seule même carte graphique NVIDIA H100 NVL 94GB a été utilisée tout au long du projet afin de permettre une comparaison de consommation de ressources.

4.3.2 Description et Prétraitement des jeux de données pour LogLLM

Les jeux de données sélectionnés n'étaient pas tous immédiatement compatibles avec LogLLM pour le benchmarking. Un prétraitement a donc été nécessaire afin de les adapter au format requis et d'optimiser les performances du modèle.

HDFS_v1 HDFS_v1 est largement utilisé dans la littérature pour le benchmarking de solutions de détection d'anomalies. Il était directement utilisé dans le benchmarking des études de LogAI et LogLLM. Aucun formatage n'a dû être réalisé pour HDFS_v1.

HDFS_v1 est un jeu de données de logs de système de fichier provenant du Hadoop Distributed File System (Shvachko et al., 2010) qui est un système de fichiers distribué conçu pour stocker de grands volumes de données. Les fichiers sont découpés en blocs et chaque bloc est répliqué sur plusieurs serveurs pour la tolérance aux pannes.
HDFS_v1 a été généré en utilisant des charges de travail pour benchmark afin de simuler différentes opérations typiques sur le système de fichiers. Les logs ont été enregistrés sur une durée de 38.7 heures pour un total de 1.5GB. Les logs collectés sont répartis en séquences en fonction de leur identifiant de bloc et chaque séquence reçoit une annotation binaire (normal ou anomalie). L'annotation a été réalisée de manière manuelle à l'aide de règles statiques afin d'identifier les anomalies. Les anomalies présentent ne sont pas des cyberattaques mais des défaillances systèmes et des erreurs critiques (Xu et al., 2009; Zhu et al., 2023).

CIC_UNSW_NB15 CIC_UNSW_NB15 (Canadian Institute for Cybersecurity, 2015) est un jeu de données d'intrusions réseaux paru en 2024 suite à une amélioration réalisée par le Canadian Institute for Cybersecurity (CIC) sur le jeu de données UNSW_NB15 (Nour Moustafa and Jill Slay, 2015).
UNSW_NB15 a été généré à l'aide de IXIA PerfectStorm qui est un dispositif permettant de générer du trafic réseau (Keysight, 2025). Le jeu de données généré synthétiquement par le dispositif inclut du trafic réseau moderne normal et anormal. Parmi le trafic anormal, neuf catégories d'attaques sont présentes et annotées avec leur type respectif. 100GB de trafic réseau ont été capturés en deux jours, ensuite des outils spécifiques ont été utilisés afin d'extraire 47 caractéristiques du trafic réseau.
Afin d'améliorer le jeu de données le CIC a réalisé, à nouveau, le processus d'extraction de caractéristiques sur les mêmes données mais en utilisant un autre outil nommé CICFlowMeter (Pasakorn Tiwatthanont, 2016). Le jeu de données final contient 76 caractéristiques et fait 189MB.
Les 9 catégories d'attaques présentes sont les suivantes :

- *Fuzzers* : Envoi de données inattendues à un logiciel pour révéler des vulnérabilités.
- *Analyse* : Collecte et étude d'informations pour exploiter des vulnérabilités.
- *Backdoor* : Obtention d'un accès non autorisé via l'utilisation ou l'insertion de portes dérobées.
- *Exploit* : Exploitation de failles pour exécuter des actions non prévues sur un système.
- *Générique* : Attaques ciblant les chiffrements par blocs.
- *Reconnaissance* : Collecte d'informations sur un système pour découvrir ses vulnérabilités sans impact direct.
- *Shellcode* : Injection de code malveillant via des vulnérabilités pour prendre le contrôle d'un système.
- *Vers* : Propagation autonome de logiciels malveillants à travers les réseaux.
- *Déni de service (Deny of Service, DoS)* : Saturation d'un service ou système pour le rendre indisponible aux utilisateurs légitimes.

Afin d'utiliser CIC_UNSW_NB15 avec LogLLM un prétraitement a été réalisé via le script : *script_perso_dataset_concat_multilabel.py*. Ce dernier permet de concaténer les jeux de données, initialement séparés, des annotations avec celui des autres caractéristiques. Il convertit également l'annotation multi-classe de CIC_UNSW_NB15 en annotation binaire. Pour ce faire, il regroupe toutes les attaques dans la classe anormale car LogLLM est uniquement prévu pour la classification binaire.

CSE_CIC_IDS2018 Le jeu de données CSE_CIC_IDS2018 (Canadian Institute for Cybersecurity, 2018) a été réalisé lors d'une collaboration entre le Communications Security Establishment (CSE) et le CIC. C'est un jeu comportant des activités normales ainsi que des intrusions réseau annotées selon leur catégorie. Les données sont tirées de captures de trafic réseau. Les logs d'événements Windows et Linux correspondants sont également disponibles dans des fichiers séparés.

La génération du jeu de données a été réalisée à l'aide de deux approches distinctes :

1. Les activités bénignes ont été produites de manière synthétique. Pour ce faire, le profil type d'utilisateurs humains non malicieux a été extrait en appliquant des méthodes de

ML et des analyses statistiques aux événements réseau qu'ils produisaient. Puis un agent (*CIC-BenignGenerator*) a reproduit un trafic réseau conforme à ce profil.

2. Pour les activités malicieuses, des attaques ont été simulées sur une infrastructure réelle créée pour l'occasion, comportant 50 machines d'attaque et une cible de 420 postes et 30 serveurs, répartis sur cinq sous-réseaux et utilisant des systèmes d'exploitation variés (Windows 8.1, 10, Server 2012/2016, Ubuntu).

Le trafic réseau a été capturé au format PCAP puis 79 caractéristiques ont été extraites avec CICFlowMeter-V3 (Pasakorn Tiwatthanont, 2016) afin de constituer des jeux de données journaliers, chacun mettant en scène une ou plusieurs attaques différentes.
Les catégories d'attaques et les spécificités de leur réalisation étaient les suivantes :

- *Infiltration :* Un fichier malveillant est envoyé par e-mail à la victime et une vulnérabilité d'application est exploitée. Une porte dérobée est exécutée sur l'ordinateur de la victime puis son ordinateur est utilisé pour scanner le réseau interne à la recherche d'autres machines vulnérables. Les machines vulnérables détectées sont exploitées à leur tour lorsque possible.
- *DoS :* Quatre outils, chacun dédié à une attaque spécifique, sont employés pour saturer les ressources des serveurs web et les rendre indisponibles.
- *Attaque contre les applications web :* L'application cible est d'abord analysée avec un scanner de vulnérabilités, puis divers types d'attaques sont effectués, dont des injections SQL (Structured Query Language), des injections de commandes, des téléchargements de fichiers non restreints, des attaques par force brute et des attaques Cross-Site Scripting.
- *Force brute :* Ces attaques visent à compromettre des comptes en automatisant de nombreuses tentatives d'accès. Des attaques par force brute utilisant un dictionnaire (liste de mots de passe courants) ont été lancées afin d'obtenir différents accès. Les attaques étaient séparées selon le type d'accès ciblé : SSH, File Transfer Protocol ou MySQL.
- *Déni de service distribué (Distributed Denial of Service, DDoS) :* Dans ce scénario, quatre ordinateurs envoient un grand nombre de requêtes vers un site web cible afin de le saturer et de le rendre inaccessible. Trois types d'attaques distincts sont réalisés, utilisant plusieurs outils et générant différents types de requêtes.

- *Botnet :* Les ordinateurs infectés ont été transformés en bots contrôlables à distance. Après infection, des captures d'écran étaient régulièrement demandées aux bots à intervalles de 400 secondes.

Deux tentatives d'amélioration de CSE_CIC_IDS2018 ont été réalisées : la version CSE_CIC_IDS2018 improved (Liu, Lisa et al., 2022) basée sur un échantillonnage plus fin, et la version BCCC-CSE-CIC-IDS2018 (Shafi et al., 2025a) utilisant un extracteur de caractéristiques avancé. Les deux ont été testées mais non retenues en raison de l'augmentation excessive de la taille du jeu de données.

Afin d'utiliser CSE_CIC_IDS2018 avec LogLLM, des prétraitements ont été réalisés.
Le jeu de données CSE_CIC_IDS2018 est initialement structuré par jour, chaque fichier pouvant contenir une ou plusieurs catégories d'attaques. Afin de le rendre compatible avec LogLLM, plusieurs étapes de prétraitement ont été réalisées à l'aide de commandes shell Linux ou de scripts Python :

1. `Séparation par catégorie d'attaque`

   Six ensembles de données ont été créés, chacun correspondant à une catégorie générale d'attaque :

   - Force brute
   - Bot
   - DDoS
   - DoS
   - Attaque Web
   - Infiltration

   Lorsque des attaques de la même catégorie étaient présentes dans plusieurs fichiers journaliers, ceux-ci ont été concaténés à l'aide d'un script Python *script_perso_concat.py*.

2. `Nettoyage et traitement des horodatages`

   Les fichiers journaliers n'étaient pas toujours ordonnés chronologiquement et utilisaient un format 12 heures pour les horodatages (par exemple, 14h était noté 02h). Deux commandes shell Linux ont été utilisées :

- La première pour convertir les horodatages au format 24 heures ;
- La seconde pour trier les entrées en fonction de l'ordre temporel.

Certains fichiers contenaient des valeurs aberrantes ou erronées dans les horodatages. Ces lignes ont été identifiées et supprimées à l'aide de deux commandes shell supplémentaires.

3. `Harmonisation des colonnes`

   Un des fichiers comportait des colonnes de caractéristiques supplémentaires. Pour garantir l'uniformité structurelle des ensembles de données et permettre une concaténation correcte, ces colonnes ont été supprimées par commande shell.

4. `Transformation des annotations`

   L'annotation du jeu de données original étant de type multi-classe, une commande shell a été appliquée à chaque ensemble afin de la convertir en annotation binaire. Tous les types d'attaques ont été remplacés par une classe unique indiquant un comportement anormal. Toutefois, l'information relative à la catégorie spécifique de l'attaque reste conservée, puisque chaque fichier correspond à une catégorie distincte.

5. `Génération des ensembles d'entraînement et de test`

   Une version modifiée du script Python *script_perso_window.py*, appelée *script_perso_window_randomsplit.py*, a été développée pour générer les ensembles finaux *train.csv* et *test.csv*. Ce script assure les étapes suivantes :

   - Génération des fenêtres temporelles,
   - Mélange aléatoire des échantillons fenêtrés,
   - Séparation du jeu de données en ensembles d'entraînement et de test.

   La modification principale apportée dans cette version est le mélange aléatoire des fenêtres temporelles avant la séparation. Une division purement chronologique engendrait des écarts importants entre les phases d'attaque présentes dans l'ensemble d'entraînement et celles de l'ensemble de test. Le mélange a donc permis d'augmenter la diversité des attaques dans l'ensemble d'entraînement, améliorant les performances du modèle.

LANL Cybersecurity Dataset Le LANL Cybersecurity Dataset (Kent, 2015), développé par le Los Alamos National Laboratory, couvre 58 jours consécutifs d'activité réseau sur

leur infrastructure interne. Il contient des événements horodatés générés par 12 425 utilisateurs et 17 684 ordinateurs. Il a l'avantage de contenir des traces de sources très variées. Les données sont réparties dans plusieurs fichiers textes, chacun représentant une source différente d'événements :

- *auth.txt* : événements d'authentification Windows provenant de postes utilisateurs, de serveurs et de contrôleurs de domaine Active Directory.
- *proc.txt* : création et arrêt de processus sur les postes Windows.
- *flows.txt* : trafic réseau capturé depuis les routeurs centraux.
- *dns.txt* : requêtes DNS enregistrées sur le serveur DNS interne.
- *redteam.txt* : traces d'attaques simulées par une équipe Red Team. Ces attaques correspondent à des authentifications réalisées avec des identifiants compromis. Chaque événement est horodaté (temps anonymisé en secondes depuis le début de la collecte) et décrit par : utilisateur@domaine, machine source et machine de destination.

Afin de rendre ce jeu de données compatible avec LogLLM, un CSV unifié a été produit en plusieurs étapes.

1. **Ajout des en-têtes :** À l'aide de commandes shell Linux, des lignes d'en-têtes ont été ajoutées à chacun des fichiers en se basant sur la documentation officielle du LANL (Kent, 2015).
2. **Traitement principal avec LANL.py :** Un script Python nommé *LANL.py* a été développé et exécuté. Il permet d'effectuer les actions suivantes :
   i. `Annotation` des fichiers *auth.txt*, *proc.txt*, *flows.txt* et *dns.txt* en comparant chaque ligne avec les événements anormaux contenus dans *redteam.txt*. Un événement est marqué comme anormal si toutes ses caractéristiques pertinentes correspondent à un événement Red Team (avec une tolérance de ±10 secondes sur l'horodatage). Les colonnes comparées varient selon le fichier source.
   ii. `Concaténation` des quatre fichiers annotés dans un seul fichier CSV, avec une colonne supplémentaire `type` indiquant la source de chaque événement.
   iii. `Tri chronologique` du fichier CSV.
   iv. `Enregistrement` du fichier final sous le nom *LANL.csv*.

3. **Réduction des noms de colonnes :** Une commande shell a ensuite été utilisée pour raccourcir les noms des colonnes afin de minimiser l'espace occupé dans les fichiers *train.csv* et *test.csv*, tout en maintenant une lisibilité suffisante pour LogLLM.
4. **Réduction du volume de données normales :** Le jeu de données étant extrêmement volumineux et comportant une écrasante majorité d'événements normaux, plusieurs techniques ont été appliquées afin de supprimer des données de classe normale le plus tôt possible dans le processus de formatage et d'économiser des ressources.
   i. `Fenêtrage optimisé` : Le script Python *script_perso_window_nan.py* a été utilisé en remplacement du script standard *script_perso_window.py*. Il introduit les améliorations suivantes : Optimisation de l'utilisation de la mémoire lors de la génération des ensembles *train.csv* et *test.csv* via fenêtrage temporel ; Ignore des séquences initiales et finales connues comme normales, afin de réduire la taille du fichier traité ; Suppression automatique des colonnes contenant uniquement des valeurs vides après la conversion des données au format texte. Cette opération évite une perte d'espace inutile sans compromettre l'intégrité des lignes, dont les autres colonnes pertinentes sont conservées.
   ii. `Réduction aléatoire des séquences normales` : À l'aide du script Python *script_perso_normal_reduction.py*, un échantillonnage aléatoire a été appliqué aux séquences normales : seule une séquence sur 5000 a été conservée. Cette opération a été effectuée uniquement sur les fichiers *train.csv* et *test.csv*, après fenêtrage, et non sur *LANL.csv*. Afin de garantir l'intégrité de chaque séquence temporelle.

4.4 Tentatives d'amélioration de LogLLM

4.4.1 Augmentation du nombre d'epochs

L'un des hyperparamètres (réglage fixé avant l'entraînement) clés est le nombre d'epochs, soit le nombre de passes complètes du modèle sur les données d'entraînement. Par défaut, LogLLM entraîne l'ensemble de son architecture (BERT, Projecteur, Llama) sur 3 epochs. Cependant, en règle générale et notamment pour LogAI (Cheng et al., 2023), les modèles sont souvent entraînés avec un nombre d'epochs bien plus élevé, permettant un affinement plus poussé des poids et, potentiellement, de meilleures performances. Une série de tests a donc

été menée avec une variation du nombre d'epochs de 1 à 30, dans le but d'évaluer l'impact de ce paramètre sur les performances du modèle. Ces tests ont été réalisés sur le jeu de données CIC_UNSW_NB15, sélectionné pour sa marge d'amélioration observée avec LogLLM en configuration standard.

4.4.2 Remplacement de Meta-Llama-3-8B

Depuis la publication de LogLLM, plusieurs LLMs plus récents ou spécialisés sont apparus. Une piste d'amélioration consiste à remplacer le modèle *Meta-Llama-3-8B* utilisé par défaut, par d'autres modèles compatibles basés sur la famille Llama, afin de garantir une intégration technique fluide. Deux modèles ont été sélectionnés pour cette expérimentation : ils reposent tous deux sur *Llama-3.1-8B*, une version plus récente que celle utilisée par LogLLM.

- `Llama-Primus-Merged` (Trend Micro, 2025; Yu et al., 2025) : basé sur *Llama-3.1-8B-Instruct*, ce modèle a été fine-tuné sur un corpus spécialisé en cybersécurité. Il bénéficie ainsi d'une meilleure compréhension de la syntaxe des logs et des flux réseau, tout en conservant une taille équivalente au modèle original de LogLLM.
- `DeepSeek-R1-Distill-Llama-8B` (DeepSeek, 2025a) : il s'agit d'un modèle distillé, c'est-à-dire entraîné à partir des générations d'un modèle plus volumineux (en l'occurrence, *DeepSeek-R1* (DeepSeek, 2025b)). Ce procédé permet d'améliorer l'efficacité et les performances du modèle final, tout en maintenant une taille réduite.

Ces deux variantes de LogLLM ont été testées sur trois jeux de données pour lesquels les performances du modèle de base présentaient une marge d'amélioration : *CIC_UNSW_NB15*, *CSE_CIC_IDS2018* (catégorie Attaque Web) et *LANL*.

4.4.3 Interprétation des réponses positives de LogLLM

Pour que les détections effectuées par LogLLM dépassent la simple génération de signaux binaires, il est essentiel de fournir un contexte supplémentaire permettant d'estimer la nature et la gravité des anomalies détectées. Cette approche a été explorée par Farrukh et al. (2025), qui ont utilisé un LLM à cette fin. Dans ce cadre, le script *script_perso_interpretation.py* a été développé afin de tester l'intégration d'un LLM local pour enrichir le contenu des alertes. Ce script utilise un prompt pour analyser les logs bruts à l'origine des anomalies, et génère un

commentaire contextuel à chaque détection d'anomalie. Le LLM local testé, *Llama-Primus-Merged* (Trend Micro, 2025; Yu et al., 2025), a été choisi en raison de sa spécialisation dans le domaine de la cybersécurité.

5 Résultats

5.1 Benchmarking des solutions

La table 4 recense les performances et la consommation de ressources de LogLLM sur les différents jeux de données testés.

Données	HDFS_v1	CIC_UNSW_NB15	CSE_CICIDS2018						LANL
Type d'anomalies	Défaillances et erreurs	Neuf types d'attaques	Force brute	Bot	DDos	Dos	Attaque web	Infiltration	Attaques
nbr de lignes test	115014	897	2099	2099	17996	4196	4196	1890	362
taille test	211MB	150MB	290MB	294MB	2.6GB	576MB	588MB	260MB	5.1MB
Précision	0.9923	0.9902	0.9953	0.9941	0.9979	0.9773	0.0910	0.7607	0.4634
Rappel	1.0000	0.9160	0.9968	0.9987	0.9991	0.9879	0.6893	0.6571	0.3958
F1-score	0.9961	0.9517	0.9961	0.9964	0.9985	0.9826	0.1608	0.7051	0.4270
Exactitude	0.9998	0.9085	0.9952	0.9948	0.9991	0.9793	0.8234	0.8269	0.8587
Durée (s)	2155.09	57.08	127.15	127.15	1101.32	259.30	255.32	115.15	18.02
Conso. (Wh)	51.022	1.353	2.931	2.093	21.655	6.328	11.261	2.815	0.443
Conso. (Cent.)	1.480	0.039	0.085	0.061	0.628	0.184	0.327	0.082	0.013

TABLE 4 – Comparaison des résultats de LogLLM sur différents jeux de données.

La table 5 compare les solutions LogAI et LogLLM appliquées au jeu de données HDFS_v1.

Méthode	LogAI									LogLLM
Données	HDFS_v1									
Type IA	Deeplearning						Machine Learning			LLM
Model	CNN		LSTM		Transformer		One_Class_SVM	Isolation_Forest	LOF	Bert et Llama
Supervision ?	✓	x	✓	x	✓	x	x	x	x	✓
Précision	0.8476	0.8740	0.8642	0.8188	0.8016	0.8036	0.0192	0.0336	0.0247	0.9923
Rappel	0.9603	0.6691	0.9894	0.7870	0.8781	0.4489	0.1311	0.4154	0.9106	1.0000
F1-score	0.9004	0.7579	0.9226	0.8026	0.8381	0.5760	0.0335	0.0621	0.0481	0.9961
Exactitude	0.9919	0.9407	0.9937	0.9464	0.9870	0.9086	0.8051	0.6763	0.0714	0.9998
Durée (s)	28.79	58.58	28.88	60.50	29.81	60.51	796.92	259.62	264.49	2155.09
Conso. (Wh)	0.716	1.434	0.707	1.484	0.742	1.484	19.833	6.447	6.564	51.022
Conso. (Cent.)	0.021	0.042	0.021	0.043	0.022	0.043	0.575	0.187	0.190	1.480

TABLE 5 – Comparaison de LogAI et LogLLM sur le jeu de données HDFS_v1.

5.2 Tentatives d'amélioration de LogLLM

5.2.1 Augmentation du nombre d'epochs

La figure 3 montre l'évolution des métriques d'évaluation (mesurées lors de la phase d'évaluation) en fonction du nombre d'epochs utilisées pendant l'entraînement de LogLLM.

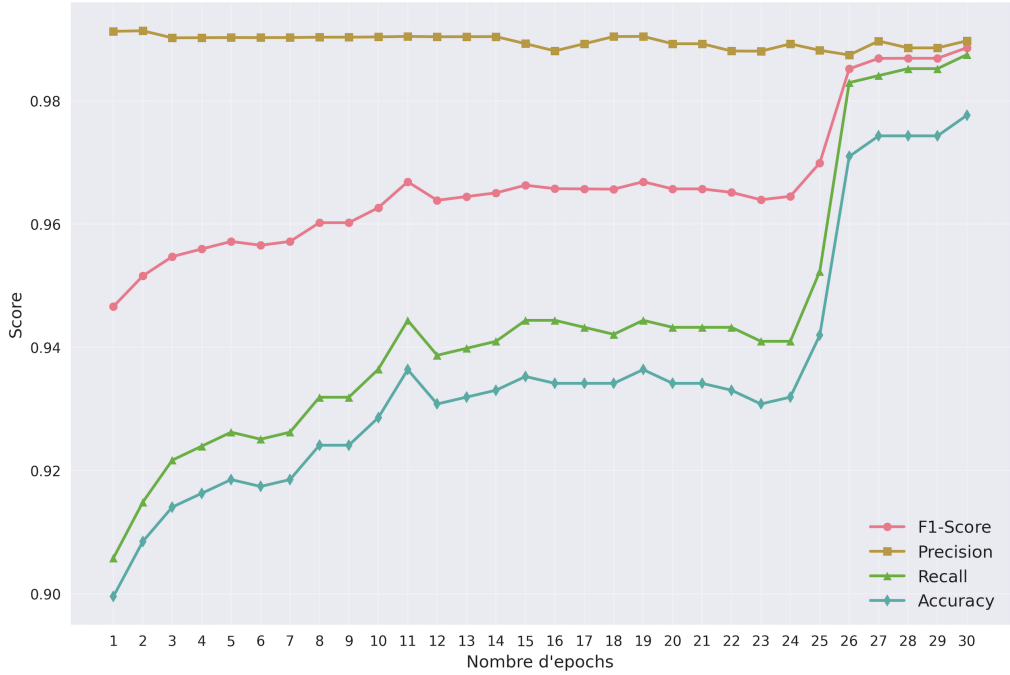

FIGURE 3 – LogLLM, Évolution des métriques d'évaluation en fonction du nombre d'epochs

5.2.2 Remplacement de Meta-Llama-3-8B

Les tables 6, 7 et 8 comparent les résultats de plusieurs variantes LogLLM sur trois jeux de données différents, respectivement : CIC_UNSW_NB15, CSE_CIC_IDS2018 Attaque Web et LANL Cybersecurity Dataset.

Version	Classique	Llama-Primus-Merged	DeepSeek-R1-Distill-Llama-8B
Jeu de données	CIC_UNSW_NB15		
Précision	0.9902	0.9945	0.9889
Rappel	0.9160	0.8263	0.9103
F1-score	0.9517	0.9027	0.9480
Exactitude	0.9085	0.8248	0.9018
Durée (s)	57.08	71.10	55.06
Conso. (Wh)	1.353	1.697	1.351
Conso. (Cent.)	0.039	0.049	0.039

TABLE 6 – Comparaison des versions de LogLLM sur le jeu de données CIC_UNSW_NB15.

Version	Classique	Llama-Primus-Merged	DeepSeek-R1-Distill-Llama-8B
Jeu de données	CSE_CIC_IDS2018 Attaque Web		
Précision	0.0910	0.0907	0.0000
Rappel	0.6893	0.8835	0.0000
F1-score	0.1608	0.1646	0.0000
Exactitude	0.8234	0.7797	0.9754
Durée (s)	255.32	253.31	216.27
Conso. (Wh)	11.261	6.167	5.276
Conso. (Cent.)	0.327	0.179	0.153

TABLE 7 – Comparaison des versions de LogLLM sur le jeu de données CSE_CIC_IDS2018 Attaque Web.

Version	Classique	Llama-Primus-Merged	DeepSeek-R1-Distill-Llama-8B
Jeu de données	LANL		
Précision	0.4634	0.3971	0.4828
Rappel	0.3958	0.5625	0.2917
F1-score	0.4270	0.4655	0.3636
Exactitude	0.8587	0.8283	0.8643
Durée (s)	18.02	19.02	18.02
Conso. (Wh)	0.443	0.464	0.445
Conso. (Cent.)	0.013	0.013	0.013

TABLE 8 – Comparaison des versions de LogLLM sur le jeu de données LANL Cybersecurity Dataset.

5.2.3 Interprétation des réponses positives de LogLLM

La table 9 affiche la pertinence de l'interprétation du modèle *Llama-Primus-Merged* selon différentes catégories d'attaques du jeu de données CSE_CIC_IDS2018. Pour les catégories composées de multiples techniques (Attaque Web et Infiltration), l'analyse se concentre sur la technique majoritaire, soit le scanne.

Catégorie d'attaque	**Interprétation correcte du LLM**
Force brute	✓
Bot	x
DDoS	✓
DoS	✓
Attaque Web (partie scanne)	✓
Infiltration (partie scanne)	✓

TABLE 9 – Pertinence de l'interprétation du LLM par catégorie d'attaque.

Le début des réponses générées par le modèle était de bonne qualité, la fin présentait fréquemment

des hallucinations, c'est-à-dire l'introduction d'éléments hors contexte non fondés sur le prompt. Par ailleurs, le modèle rencontrait des difficultés à s'arrêter correctement, prolongeant parfois la génération de texte de manière inappropriée.

6 Discussion

6.1 Benchmarking des solutions

6.1.1 Comparaison de LogLLM et des algorithmes de LogAI

Performances LogLLM démontre les meilleures performances globales sur HDFS_v1, surpassant tous les autres algorithmes sur l'ensemble des métriques : précision (0.9923), rappel (1.0000), F1-score (0.9961) et exactitude (0.9998). Viennent ensuite les méthodes de détection d'anomalies par DL proposées par LogAI : le LSTM supervisé obtient un F1-score de 0,9226, suivi du CNN supervisé (0,9004). Les versions non supervisées du CNN et du LSTM, ainsi que le Transformer supervisé, obtiennent des scores corrects (F1-score respectifs : 0.7579, 0.8026 et 0.8381). En revanche, le Transformer non supervisé affiche des résultats médiocres (F1-score 0.5760). Les méthodes de détection d'anomalies par ML proposées par LogAI (One-Class SVM, Isolation Forest, LOF) se révèlent inefficaces sur ce jeu de données, avec des F1-score extrêmement faibles : 0.0335, 0.0621 et 0.0481 respectivement. LOF sur-détecte les anomalies tandis que One-Class SVM n'en détecte qu'une faible proportion. Ces résultats peuvent s'expliquer soit par une inadéquation des méthodes à la détection d'anomalies sur HDFS, soit par des hyperparamètres mal adaptés, malgré des essais d'ajustement qui n'ont pas amélioré les performances.

Rapidité et consommation énergétique Les modèles supervisés CNN, LSTM et Transformer sont les plus rapides, avec des temps cumulatifs de vectorisation et d'inférence autour de 29 secondes. Leurs homologues non supervisés doublent le temps de traitement. Isolation Forest et LOF présentent des durées intermédiaires (environ 260 secondes), tandis que One-Class SVM est notablement plus lent (797 secondes). LogLLM reste le plus lent avec 2155 secondes.

L'analyse de la consommation électrique affiche le même classement : CNN, LSTM et Trans-

former supervisés sont les plus économes (entre 0.707 et 0.742 Wh), suivis par leurs versions non supervisées qui doublent la consommation d'énergie. Isolation Forest et LOF consomment environ 6.5 Wh, One-Class SVM atteint 19.833 Wh et LogLLM culmine à 51.022 Wh.
Il convient de noter que dans LogAI, les modèles de DL non supervisés (CNN, LSTM, Transformer) sont évalués différemment : entraînés en one-class sur des données normales, ils sont évalués sur une base de test élargie (+27 % par rapport aux autres algorithmes), incluant les anomalies exclues de l'ensemble d'entraînement. Cela impacte légèrement à la hausse la durée d'exécution et la consommation d'énergie associées à ces modèles.

6.1.2 Évaluation de LogLLM sur différents jeux de données

Performances Les résultats obtenus avec LogLLM sont prometteurs, avec d'excellentes performances sur la majorité des jeux de données.
Pour la catégorie Infiltration du jeu de données CSE_CIC_IDS2018, le F1-score atteint 0.7051. Cette performance, bien que correcte, reste inférieure à celles obtenues sur d'autres catégories. Cela s'explique en grande partie par la diversité des techniques d'attaque présentes dans cette catégorie : la plupart des exemples concernent des activités de scan, tandis que les autres techniques, beaucoup moins représentées, n'ont pas permis au modèle de bien les apprendre durant l'entraînement.
La catégorie Attaque Web du même jeu de données affiche un F1-score très faible de 0.1608, principalement en raison d'une précision particulièrement basse (0.0910). Ce résultat s'explique par un fort déséquilibre des classes, avec très peu d'anomalies disponibles pour l'entraînement. En outre, ces rares anomalies sont dispersées entre plusieurs techniques différentes, ce qui limite encore davantage le nombre d'exemples pour chacune d'elles.
Plus généralement, les performances sur CSE_CIC_IDS2018 sont limitées par des problèmes de qualité d'annotation du jeu de données, comme le soulignent Liu, Lisa et al. (2022).
Enfin, sur le jeu de données LANL, LogLLM obtient un F1-score de 0,4267, une performance certes modeste mais nettement supérieure à l'aléatoire. Cette limitation s'explique par la rareté des anomalies dans ce jeu de données, qui offre peu d'exemples pertinents pour l'entraînement. Des informations contextuelles supplémentaires sur les attaques Red Team auraient également permis une annotation plus optimale durant cette étude.

Rapidité et consommation énergétique La durée de vectorisation et d'inférence est fortement corrélée à la consommation énergétique (Wh). L'élément ayant le plus d'impact sur le temps de traitement et la consommation d'énergie n'est pas la taille totale du jeu de données, mais le nombre de séquences à traiter. Cette observation s'explique par l'architecture de LogLLM, qui sollicite Llama une fois pour chaque séquence de texte brut. Ainsi, c'est principalement le nombre d'appels à Llama, plutôt que la taille des séquences textuelles, qui conditionne la rapidité et la consommation énergétique du processus.

6.2 Tentatives d'amélioration de LogLLM

6.2.1 Augmentation du nombre d'epochs

L'augmentation du nombre d'epochs pendant l'entraînement de LogLLM se traduit par une amélioration progressive des métriques d'évaluation (lors de la phase d'évaluation), en particulier du rappel et de l'exactitude. La précision, déjà élevée dès la première epoch, n'augmente que légèrement. Les meilleures performances sont atteintes à l'epoch 30, laissant envisager des gains supplémentaires si l'entraînement était poursuivi au-delà. D'autres hyperparamètres comme la taille des fenêtres temporelles ou le taux d'apprentissage (vitesse à laquelle un modèle ajuste ses poids lors de l'entraînement) pourraient également être ajustés afin d'améliorer les performances de la solution.

6.2.2 Remplacement de Meta-Llama-3-8B

L'évaluation des modèles *Llama-Primus-Merged* et *DeepSeek-R1-Distill-Llama-8B* n'a pas mis en évidence d'amélioration systématique des performances par rapport au modèle classique, *Meta-Llama-3-8B*.

Le modèle *Llama-Primus-Merged* améliore le F1-score sur les jeux de données CSE_CIC_IDS2018 et LANL, mais affiche une performance inférieure sur CIC_UNSW_NB15. Bien qu'il soit généralement plus lent que la version classique, sa consommation énergétique varie selon le jeu de données, sans être systématiquement plus élevée.

De son côté, *DeepSeek-R1-Distill-Llama-8B* montre une instabilité notable, particulièrement sur CSE_CIC_IDS2018 où il procède à une classification entièrement normale, conduisant à une exactitude élevée mais à un F1-score nul. Ce modèle enregistre une baisse systématique

du F1-score par rapport à la version classique. Il présente des temps d'inférence plus courts et une consommation énergétique globalement réduite.
Il serait également pertinent d'explorer d'autres LLMs qui ne sont pas basés sur Llama, ou d'évaluer des versions plus volumineuses, telles que *Meta-Llama-3-70B* (Meta, 2024), qui offrent des capacités accrues au prix d'une consommation de ressources plus élevée.

6.2.3 Interprétation des réponses positives de LogLLM

L'interprétation des anomalies détectées par LogLLM est prometteuse mais demeure expérimentale et nécessite des améliorations pour une utilisation opérationnelle. Les réponses générées par le modèle sont pertinentes pour toutes les catégories d'attaque, à l'exception de la catégorie Bot, qui n'a pas pu être expliquée. L'intégration d'un LLM pour contextualiser les alertes apparaît donc comme une piste prometteuse. Les problèmes d'hallucinations et les prolongements textuels inappropriés pourraient être limités par l'utilisation d'un LLM plus performant, tel que *Llama-Primus-Nemotron-70B-Instruct* (Trendcybertron, 2025), version plus puissante de *Llama-Primus-Merged*. Enfin, l'adoption d'un gestionnaire d'appels LLM local, comme Ollama (Ollama, 2025), faciliterait l'intégration du modèle d'interprétation dans la chaîne de traitement.

6.3 Cas d'utilisation du modèle : Forensique vs SOC

Bien que ce travail ait été orienté vers le développement d'un modèle de détection d'anomalies pour un usage en SOC, ce type de modèle peut également présenter un intérêt dans un contexte forensique. Le choix du modèle d'IA doit toutefois être adapté à l'application visée. En SOC, la détection en temps réel est primordiale afin de permettre une réaction rapide et de limiter les effets d'une attaque : la rapidité d'inférence et la faible consommation énergétique deviennent alors des critères déterminants, compte tenu de l'exigence de disponibilité continue (24/7) et de l'impact énergétique associé. À l'inverse, la forensique vise à analyser des incidents a posteriori pour reconstituer les événements de manière ponctuelle. Dans ce cas, la rapidité d'inférence demeure importante mais moins contraignante, la consommation énergétique est secondaire et la priorité va à la performance.

6.4 Propositions de développements futurs

6.4.1 Évolutions possibles : modèles spécialisés et approches d'ensemble

Ce travail s'est concentré sur un modèle généraliste, capable de traiter une grande diversité de sources et de typologies de traces. Une piste d'évolution consisterait à utiliser plusieurs modèles spécialisés, chacun dédié à une source ou un type de données spécifique. Par exemple, un modèle pourrait être déployé pour chaque index de données stocké sur Opensearch, permettant de générer des alertes d'anomalie individualisées et de fournir une information sur la source concernée par l'anomalie. Cette approche s'inspire notamment de la solution propriétaire Vectra AI, qui regroupe environ 150 modèles spécialisés, chacun entraîné sur une technique particulière du MITRE ATT&CK (Vectra, 2025). Une complémentarité à l'utilisation isolée de modèles spécialisés serait de recourir à des méthodes d'ensemble. Les techniques d'ensemble consistent à combiner plusieurs modèles, dits faibles, pour obtenir de meilleures performances qu'avec un modèle unique. Chaque sous-modèle spécialisé opère sur un sous-ensemble de données et produit sa propre prédiction ; ces prédictions sont ensuite agrégées, parfois à l'aide d'un méta-modèle comme c'est le cas avec la méthode du stacking, afin d'établir une décision finale (Soni, 2023). Cette approche permettrait de conserver une vision globale et multi-source du système surveillé, prenant en compte d'éventuelles corrélations entre différentes sources de traces. À titre d'illustration, Liu et al. (2024) ont proposé une méthode d'ensemble destinée à améliorer la généralisation et par conséquent l'utilité de la détection d'anomalies dans des contextes d'utilisation réels.

6.4.2 Fonctionnalités supplémentaires

D'autres fonctionnalités pourraient renforcer la solution. Il serait pertinent d'ajouter une interface graphique pour faciliter la détection d'anomalies, l'affichage et l'interprétation des alertes. Par ailleurs, la mise en place de mécanismes de notification, tels que l'envoi d'alertes par e-mail ou SMS, seraient également bénéfiques.

7 Conclusion

L'objectif de ce travail était de permettre la conception d'une solution de détection d'anomalies basée sur l'IA, performante et économe en énergie, adaptée à l'environnement de SecuLabs et SeLabs. Pour cela, un cadre de conception de la future solution SOC-IA a été défini, permettant d'anticiper la nature des données disponibles, l'architecture technique utilisée et la chaîne de traitement envisagée. Un état de l'art des solutions d'IA existantes et des jeux de données pertinents a été réalisé, suivi d'une évaluation comparative en termes de performances et de consommation énergétique pour LogLLM et LogAI. Des pistes d'amélioration de LogLLM ont également été explorées et évaluées.

La future solution SOC-IA reposera sur la collecte et l'analyse automatisée de traces hétérogènes provenant de l'environnement du client, s'appuyant sur une infrastructure sécurisée en Suisse et une chaîne de traitement adaptée. L'entraînement des modèles tiendra compte de l'activité normale propre à chaque client tout en s'appuyant sur des jeux de données riches et réels obtenus via le partenariat avec l'organisation x.

Parmi toutes les solutions IA libres d'accès revues, LogLLM et LogAI ont été retenues pour leur potentiel et leur chaîne de traitement complète, intégrant parsing, prétraitement, apprentissage et inférence. Les benchmarks montrent que LogLLM surpasse largement les autres méthodes en termes de performance, mais reste très coûteux en temps de traitement et en consommation énergétique. Ses performances varient selon la qualité et l'équilibre des jeux de données, soulignant l'importance de disposer de données de qualité, riches en exemples d'anomalie correctement annotées. Les modèles de DL classiques intégrés dans LogAI sont plus rapides et économes et restent performants, les plus performants en termes de F1-score sont, dans l'ordre, LSTM supervisé, CNN supervisé, Transformer supervisé, LSTM non supervisé. Les modèles de ML proposés par LogAI se sont eux révélés inefficaces dans ce contexte.

Les tentatives d'amélioration de LogLLM ont révélé un potentiel d'amélioration, notamment par l'ajustement des hyperparamètres, tels que l'augmentation du nombre d'epochs d'entraînement. En revanche, le remplacement de Meta-Llama-3-8B par un autre LLM n'a pas permis d'obtenir une amélioration de performances systématique. Par ailleurs, l'interprétation des prédictions positives de LogLLM, en vue de générer des alertes contextualisées, s'est révélée prometteuse mais reste à consolider pour un usage opérationnel.

Pour l'avenir, il serait pertinent d'approfondir le recours à plusieurs modèles spécialistes et à des méthodes d'ensemble. L'ajout de fonctionnalités, telles qu'une interface graphique pour la visualisation et l'interprétation des alertes, ainsi que des systèmes de notification automatisée, renforcerait l'utilisabilité de la solution.

Références

Adam Oliner and Jon Stearley. What Supercomputers Say : A Study of Five System Logs. In *37th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN'07)*, pages 575–584, Édimbourg, Royaume-Uni, 2007. doi : 10.1109/DSN.2007.103.

Afshine Amidi and Shervine Amidi. CS 230 - Recurrent Neural Networks Cheatsheet. https ://stanford.edu/˜shervine/teaching/cs-230/cheatsheet-recurrent-neural-networks#overview, 2025.

R. Allen. Active Directory Logs : Monitor AD Security and Performance, July 2022.

L. Alzubaidi, J. Zhang, A. J. Humaidi, A. Al-Dujaili, Y. Duan, O. Al-Shamma, J. Santamaría, M. A. Fadhel, M. Al-Amidie, and L. Farhan. Review of deep learning : Concepts, CNN architectures, challenges, applications, future directions. *Journal of Big Data*, 8(1) :53, Mar. 2021. ISSN 2196-1115. doi : 10.1186/s40537-021-00444-8.

Amit Galitzky. GitHub - opensearch-project/anomaly-detection. https ://github.com/opensearch-project/anomaly-detection ?tab=readme-ov-file#opensearch-anomaly-detection, 2025.

S. S. Bamber, A. V. R. Katkuri, S. Sharma, and M. Angurala. A hybrid CNN-LSTM approach for intelligent cyber intrusion detection system. *Computers & Security*, 148(104146), Jan. 2025. ISSN 01674048. doi : 10.1016/j.cose.2024.104146.

Canadian Institute for Cybersecurity. NSL-KDD | Datasets | Research | Canadian Institute for Cybersecurity | UNB. https ://www.unb.ca/cic/datasets/nsl.html, 2009.

Canadian Institute for Cybersecurity. UNSW-NB15 Augmented Dataset | Datasets | Research

| Canadian Institute for Cybersecurity | UNB. https ://www.unb.ca/cic/datasets/cic-unsw-nb15.html, 2015.

Canadian Institute for Cybersecurity. IDS 2018 | Datasets | Research | Canadian Institute for Cybersecurity | UNB. https ://www.unb.ca/cic/datasets/ids-2018.html, 2018.

M. Cantone, C. Marrocco, and A. Bria. On the Cross-Dataset Generalization of Machine Learning for Network Intrusion Detection. 2024. ISSN 2169-3536. doi : 10.48550/arXiv.2402.10974.

Z. Chen, J. Liu, W. Gu, Y. Su, and M. R. Lyu. Experience Report : Deep Learning-based System Log Analysis for Anomaly Detection, Jan. 2022.

Q. Cheng, A. Saha, W. Yang, C. Liu, D. Sahoo, and S. Hoi. LogAI : A Library for Log Analytics and Intelligence, Jan. 2023.

Conor O'Sullivan. Isolation Forest Guide : Explanation and Python Implementation. https ://www.datacamp.com/tutorial/isolation-forest, 2024.

G. Creech. *Developing a High-Accuracy Cross Platform Host-Based Intrusion Detection System Capable of Reliably Detecting Zero-Day Attacks*. Thesis, UNSW Sydney, 2014.

G. Creech and J. Hu. A Semantic Approach to Host-Based Intrusion Detection Systems Using Contiguousand Discontiguous System Call Patterns. *IEEE Transactions on Computers*, 63 (4) :807–819, Apr. 2014. ISSN 1557-9956. doi : 10.1109/TC.2013.13.

DeepSeek. Deepseek-ai/DeepSeek-R1-Distill-Llama-8B · Hugging Face. https ://huggingface.co/deepseek-ai/DeepSeek-R1-Distill-Llama-8B, May 2025a.

DeepSeek. Deepseek-ai/DeepSeek-R1 · Hugging Face. https ://huggingface.co/deepseek-ai/DeepSeek-R1, May 2025b.

P. D. Domański. Study on Statistical Outlier Detection and Labelling. *International Journal of Automation and Computing*, 17(6) :788–811, Dec. 2020. ISSN 1751-8520. doi : 10.1007/s11633-020-1243-2.

M. Du, F. Li, G. Zheng, and V. Srikumar. DeepLog : Anomaly Detection and Diagnosis from System Logs through Deep Learning. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, CCS '17, pages 1285–1298, New York, NY, USA, Oct. 2017. Association for Computing Machinery. ISBN 978-1-4503-4946-8. doi : 10.1145/3133956.3134015.

Elasticsearch. Kibana Dashboard. https ://www.elastic.co/kibana/kibana-dashboard, 2025.

Y. A. Farrukh, S. Wali, I. Khan, and N. D. Bastian. XG-NID : Dual-Modality Network Intrusion Detection using a Heterogeneous Graph Neural Network and Large Language Model, May 2025.

M. A. Ferrag, O. Friha, D. Hamouda, L. Maglaras, and H. Janicke. Edge-IIoTset : A New Comprehensive Realistic Cyber Security Dataset of IoT and IIoT Applications for Centralized and Federated Learning. 10 :40281–40306, Jan. 2022.

FluentBit. Fluent Bit Documentation. https ://docs.fluentbit.io/manual, June 2025.

R. Foorthuis. A Typology of Data Anomalies. In *Information Processing and Management of Uncertainty in Knowledge-Based Systems. Theory and Foundations*, volume 854, pages 26–38. Springer International Publishing, 2018. doi : 10.1007/978-3-319-91476-3_3.

Fortinet. Qu'est-ce que l'analyse des entités et du comportement des utilisateurs (UEBA) ? https ://www.fortinet.com/fr/resources/cyberglossary/what-is-ueba.html, 2025.

Gideon Creech and Jiankun Hu. Generation of a new IDS test dataset : Time to retire the KDD collection. In *ResearchGate*, pages 4487–4492, Shanghai, Chine, 2013. doi : 10.1109/WCNC.2013.6555301.

Google. Google-bert/bert-base-uncased · Hugging Face. https ://huggingface.co/google-bert/bert-base-uncased, Mar. 2024.

Google. Présentation de la structure du GAN | Machine Learning. https ://developers.google.com/machine-learning/gan/gan_structure ?hl=fr, Feb. 2025.

W. Guan, J. Cao, S. Qian, J. Gao, and C. Ouyang. LogLLM : Log-based Anomaly Detection Using Large Language Models, Feb. 2025.

H. Guo, S. Yuan, and X. Wu. LogBERT : Log Anomaly Detection via BERT, Mar. 2021.

S. Guthula, R. Beltiukov, N. Battula, W. Guo, A. Gupta, and I. Monga. netFound : Foundation Model for Network Security, Jan. 2025.

X. Han, S. Yuan, and M. Trabelsi. LogGPT : Log Anomaly Detection via GPT, Dec. 2023.

S. He, J. Zhu, P. He, and M. R. Lyu. Experience Report : System Log Analysis for Anomaly Detection. In *2016 IEEE 27th International Symposium on Software Reliability Engineering (ISSRE)*, pages 207–218, Ottawa, ON, Canada, Oct. 2016. IEEE. doi : 10.1109/issre.2016.21.

M. Hesamodin, Arash Habibi Lashkari, and Ali A. Ghorbani. Poisoning and Evasion : Deep Learning-Based NIDS under Adversarial Attacks | Request PDF. In *ResearchGate*, Sydney, Australie, Aug. 2024. doi : 10.1109/PST62714.2024.10788064.

Hettich, S and Bay, S. D. KDD Cup 1999 Data. https ://kdd.ics.uci.edu/databases/kddcup99/kddcup99.html ?utm_source=openai, Oct. 1999.

K. Highnam, K. Arulkumaran, Z. Hanif, and N. R. Jennings. BETH Dataset : Real Cybersecurity Data for Unsupervised Anomaly Detection Research. In *Conference on Applied Machine Learning for Information Security*, Virginia, États-Unis, 2021.

A. V. Joshi. Introduction to AI and ML. In A. V. Joshi, editor, *Machine Learning and Artificial Intelligence*, pages 3–7. Springer International Publishing, Cham, 2020. ISBN 978-3-030-26622-6. doi : 10.1007/978-3-030-26622-6_1.

K Kent and M P Souppaya. Guide to computer security log management. Technical report, National Institute of Standards and Technology, Gaithersburg, MD, 2006.

R. Kalra. Introduction to Transformers and Attention Mechanisms. https ://medium.com/@kalra.rakshit/introduction-to-transformers-and-attention-mechanisms-c29d252ea2c5, Feb. 2024.

R. Kassel. Graph Neural Networks (GNN) : qu'est-ce que c'est ?, Apr. 2023.

A. D. Kent. Cybersecurity Data Sources for Dynamic Network Research. In *Dynamic Networks in Cybersecurity*, pages 37–65. Imperial College Press, June 2015.

Keysight. PerfectStorm. https ://www.keysight.com/us/en/products/network-test/network-test-hardware/perfectstorm.html, 2025.

D. Khurana, A. Koli, K. Khatter, and S. Singh. Natural Language Processing : State of The Art, Current Trends and Challenges. *Multimedia Tools and Applications*, 82(3) :3713–3744, Jan. 2023. ISSN 1380-7501, 1573-7721. doi : 10.1007/s11042-022-13428-4.

Kinza Yasar and Alexander S. Gillis. What is Windows Event Log ? | Definition from TechTarget. https ://www.techtarget.com/searchwindowsserver/definition/Windows-event-log, Mar. 2023.

V.-H. Le and H. Zhang. Log-based Anomaly Detection Without Log Parsing, Aug. 2021.

Y. LeCun, Y. Bengio, and G. Hinton. Deep learning. *Nature*, 521(7553) :436–444, May 2015. ISSN 1476-4687. doi : 10.1038/nature14539.

Z. Li, J. Shi, and M. van Leeuwen. Graph Neural Networks based Log Anomaly Detection and Explanation, Jan. 2024.

Q. Lin, H. Zhang, J.-G. Lou, Y. Zhang, and X. Chen. Log clustering based problem identification for online service systems. In *Proceedings of the 38th International Conference on Software Engineering Companion*, ICSE '16, pages 102–111, New York, NY, USA, May 2016. Association for Computing Machinery. ISBN 978-1-4503-4205-6. doi : 10.1145/2889160.2889232.

Y. Lin, H. Deng, and X. Li. FastLogAD : Log Anomaly Detection with Mask-Guided Pseudo Anomaly Generation and Discrimination, Apr. 2024.

Y. Liu, L. Zhu, L. Ding, Z. Huang, H. Sui, S. Wang, and Y. Song. Selective ensemble method for anomaly detection based on parallel learning. *Scientific Reports*, 14(1420), Jan. 2024. ISSN 2045-2322. doi : 10.1038/s41598-024-51849-3.

Liu, Lisa, Engelen, Gints, Lynar, Timothy, Essam, Daryl, and Joosen, Wouter. Error Prevalence in NIDS datasets : A Case Study on CIC-IDS-2017 and CSE-CIC-IDS-2018. In *2022*

IEEE Conference on Communications and Network Security (CNS), pages 254–262, Austin, Texas, États-Unis, 2022. IEEE.

L. D. Manocchio, S. Layeghy, W. W. Lo, G. K. Kulatilleke, M. Sarhan, and M. Portmann. FlowTransformer : A transformer framework for flow-based network intrusion detection systems. *Expert Systems with Applications*, 241 :122564, May 2024. ISSN 0957-4174. doi : 10.1016/j.eswa.2023.122564.

María del Carmen Prudente Tixteco, Lidia Prudente Tixteco, and Gabriel Sánchez Pérez. Intrusion Detection Using Indicators of Compromise Based on Best Practices and Windows Event Logs. In *The Eleventh International Conference on Internet Monitoring and Protection (ICIMP 2016)*, pages 29–37, Valence, Espagne, 2016. IARIA.

Mark Russinovich and Thomas Garnier. Sysmon - Sysinternals. https ://learn.microsoft.com/en-us/sysinternals/downloads/sysmon, July 2024.

Meta. Meta-llama/Meta-Llama-3-70B · Hugging Face. https ://huggingface.co/meta-llama/Meta-Llama-3-70B, Dec. 2024.

Meta. Meta-llama/Meta-Llama-3-8B · Hugging Face. https ://huggingface.co/meta-llama/Meta-Llama-3-8B, Dec. 2024.

Microsoft. Group Policy Objects. https ://learn.microsoft.com/en-us/previous-versions/windows/desktop/policy/group-policy-objects, May 2018.

M.-E. Mihailescu, D. Mihai, M. Carabas, M. Komisarek, M. Pawlicki, W. Hołubowicz, and R. Kozik. The Proposition and Evaluation of the RoEduNet-SIMARGL2021 Network Intrusion Detection Dataset. *Sensors*, 21(13) :4319, Jan. 2021. ISSN 1424-8220. doi : 10.3390/s21134319.

Milvus. What is the difference between rule-based and AI-based anomaly detection ? https ://milvus.io/ai-quick-reference/what-is-the-difference-between-rulebased-and-aibased-anomaly-detection, 2025.

MITRE. MITRE ATT&CK®. https ://attack.mitre.org/, 2025.

N. Moustafa and J. Slay. UNSW-NB15 : A comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set). In *2015 Military Communications and Information Systems Conference (MilCIS)*, pages 1–6, Canberra, Australia, Nov. 2015. doi : 10.1109/MilCIS.2015.7348942.

NEXA. Convolutional Neural Network (CNN), qu'est-ce que c'est ? https ://www.nexa.fr/post/convolutional-neural-network, July 2025.

Nexthink. Proactive IT Management with Nexthink. https ://www.nexthink.com/solutions/proactive-it-management, 2025.

G. No, Y. Lee, H. Kang, and P. Kang. RAPID : Training-free Retrieval-based Log Anomaly Detection with PLM considering Token-level information, Nov. 2023.

Nour Moustafa and Jill Slay. (PDF) UNSW-NB15 : A comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set). In *ResearchGate*, Canberra, Australia, Nov. 2015. doi : 10.1109/MilCIS.2015.7348942.

NVIDIA. GPU NVIDIA H100 Tensor Core. https ://www.nvidia.com/fr-fr/data-center/h100/, 2025.

Ollama. Ollama. https ://ollama.com, 2025.

Opensearch. Low-level Python client. https ://docs.opensearch.org/docs/latest/clients/python-low-level/, 2025a.

Opensearch. OpenSearch Dashboards. https ://docs.opensearch.org/docs/latest/dashboards/, 2025b.

OpenSearch. OpenSearch Documentation. https ://docs.opensearch.org/docs/latest/, 2025.

Pasakorn Tiwatthanont. iPAS / TCPDUMP_and_CICFlowMeter. https ://github.com/iPAS/TCPDUMP_and_CICFlowMeter/blob/master/CICFlowMeters/CICFlowMeter-3.0/README.md, 2016.

Patrick Kral. Incident Handler's Handbook. Technical report, SANS Institute, Dec. 2011.

Richard Gargan. Demystifying PCAP (Packet Capture) Files. https ://www.netmaker.io/resources/pcap-files, Oct. 2024.

Roberto Rodriguez. UraSecTeam/mordor. https ://github.com/UraSecTeam/mordor, 2025.

Roberto Rodriguez and Jose Luis Rodriguez. Introduction — Security Datasets. https ://securitydatasets.com/introduction.html, 2022.

Scikit-learn. Outlier detection with Local Outlier Factor (LOF). https ://scikit-learn/stable/auto_examples/neighbors/plot_lof_outlier_detection.html, 2025.

M. Shafi, A. H. Lashkari, and A. H. Roudsari. Toward Generating a Large Scale Intrusion Detection Dataset and Intruders Behavioral Profiling Using Network and Transportation Layers Traffic Flow Analyzer (NTLFlowLyzer). *Journal of Network and Systems Management*, 33(2) :44, Mar. 2025a. ISSN 1573-7705. doi : 10.1007/s10922-025-09917-0.

M. Shafi, A. H. Lashkari, and A. H. Roudsari. NTLFlowLyzer : Towards generating an intrusion detection dataset and intruders behavior profiling through network and transport layers traffic analysis and pattern extraction. *Computers & Security*, 148 :104160, Jan. 2025b. ISSN 0167-4048. doi : 10.1016/j.cose.2024.104160.

I. Sharafaldin, A. Habibi Lashkari, and A. A. Ghorbani. Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization :. In *Proceedings of the 4th International Conference on Information Systems Security and Privacy*, pages 108–116, Funchal, Madeira, Portugal, 2018. SCITEPRESS - Science and Technology Publications. ISBN 978-989-758-282-0. doi : 10.5220/0006639801080116.

A. K. Sharma, S. Patel, S. B. Wakchaure, and A. S. Enhanced Convolution Neural Network with Optimized Pooling and Hyperparameter Tuning for Network Intrusion Detection, Sept. 2024.

Shayan Hashemi and Mika Mäntylä. OneLog : Towards end-to-end software log anomaly detection | Automated Software Engineering. *Springer Nature*, 31(37), Apr. 2024. doi : 10.1007/s10515-024-00428-x.

K. Shvachko, H. Kuang, S. Radia, and R. Chansler. The Hadoop Distributed File System. In *2010 IEEE 26th Symposium on Mass Storage Systems and Technologies (MSST)*, pages 1–10, Incline Village, NV, USA, May 2010. IEEE. ISBN 978-1-4244-7152-2. doi : 10.1109/MSST.2010.5496972.

A. A. Siam, M. M. Hassan, and T. Bhuiyan. Artificial Intelligence for Cybersecurity : A State of the Art. In *2025 IEEE 4th International Conference on AI in Cybersecurity (ICAIC)*, pages 1–7, Houston, Texas, United States, Feb. 2025. doi : 10.1109/ICAIC63015.2025.10848980.

N. Smith. WraySmith/log-anomaly. https ://github.com/WraySmith/log-anomaly, Mar. 2025.

B. Soni. Stacking to Improve Model Performance : A Comprehensive Guide on Ensemble Learning in Python, May 2023.

M. Tavallaee, E. Bagheri, W. Lu, and A. A. Ghorbani. A detailed analysis of the KDD CUP 99 data set. In *2009 IEEE Symposium on Computational Intelligence for Security and Defense Applications*, pages 1–6, Ottawa, ON, Canada, July 2009. IEEE. doi : 10.1109/CISDA.2009.5356528.

Trend Micro. Trendmicro-ailab/Llama-Primus-Merged · Hugging Face. https ://huggingface.co/trendmicro-ailab/Llama-Primus-Merged, Mar. 2025.

Trendcybertron. Trend-cybertron/Llama-Primus-Nemotron-70B-Base · Hugging Face. https ://huggingface.co/trend-cybertron/Llama-Primus-Nemotron-70B-Base, Feb. 2025.

M. J. M. Turcotte, A. D. Kent, and C. Hash. Unified Host and Network Data Set, Aug. 2017.

UNSW. Next-Generation Intrusion Detection System-Dataset (NGIDS-DS), 2023.

A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin. Attention Is All You Need, Aug. 2023.

Vectra. Vectra AI - L'IA de cybersécurité qui arrête les attaques que les autres ne peuvent pas contrer. https ://fr.vectra.ai/, 2025.

Z. Wang, Z. Chen, J. Ni, H. Liu, H. Chen, and J. Tang. Multi-Scale One-Class Recurrent Neural Networks for Discrete Event Sequence Anomaly Detection, Aug. 2020.

Y. Xie, H. Zhang, and M. A. Babar. LogGD : Detecting Anomalies from System Logs with Graph Neural Networks. In *2022 IEEE 22nd International Conference on Software Quality, Reliability and Security (QRS)*, pages 299–310, Guangzhou, China, Dec. 2022. IEEE. doi : 10.1109/QRS57517.2022.00039.

Y. Xie, H. Zhang, and M. A. Babar. LogSD : Detecting Anomalies from System Logs through Self-Supervised Learning and Frequency-Based Masking. *Proc. ACM Softw. Eng.*, 1(FSE) : 93 :2098–93 :2120, July 2024. doi : 10.1145/3660800.

W. Xu, L. Huang, A. Fox, D. Patterson, and M. I. Jordan. Detecting large-scale system problems by mining console logs. In *Proceedings of the ACM SIGOPS 22nd Symposium on Operating Systems Principles*, SOSP '09, pages 117–132, New York, NY, USA, Oct. 2009. Association for Computing Machinery. ISBN 978-1-60558-752-3. doi : 10.1145/1629575.1629587.

L. Yang, J. Chen, Z. Wang, W. Wang, J. Jiang, X. Dong, and W. Zhang. PLELog : Semi-Supervised Log-Based Anomaly Detection via Probabilistic Label Estimation. In *2021 IEEE/ACM 43rd International Conference on Software Engineering : Companion Proceedings (ICSE-Companion)*, pages 230–231, May 2021. doi : 10.1109/ICSE-Companion52605.2021.00106.

H. Yao, M. Liu, Z. Yin, Z. Yan, X. Hong, and W. Zuo. GLAD : Towards Better Reconstruction with Global and Local Adaptive Diffusion Models for Unsupervised Anomaly Detection. In A. Leonardis, E. Ricci, S. Roth, O. Russakovsky, T. Sattler, and G. Varol, editors, *Computer Vision – ECCV 2024*, pages 1–17, Cham, 2025. Springer Nature Switzerland. ISBN 978-3-031-73209-6. doi : 10.1007/978-3-031-73209-6_1.

Y.-C. Yu, T.-H. Chiang, C.-W. Tsai, C.-M. Huang, and W.-K. Tsao. Primus : A Pioneering Collection of Open-Source Datasets for Cybersecurity LLM Training, June 2025.

M. Zhang, B. Xu, and J. Gong. An Anomaly Detection Model Based on One-Class SVM to Detect Network Intrusions. In *2015 11th International Conference on Mobile Ad-hoc and Sensor Networks (MSN)*, pages 102–107, Shenzhen, China, Dec. 2015. IEEE. doi : 10.1109/MSN.2015.40.

S. Zhang, Y. Liu, X. Zhang, W. Cheng, H. Chen, and H. Xiong. CAT : Beyond Efficient Transformer for Content-Aware Anomaly Detection in Event Sequences. In *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, KDD '22, pages 4541–4550, New York, NY, USA, Aug. 2022. Association for Computing Machinery. ISBN 978-1-4503-9385-0. doi : 10.1145/3534678.3539155.

J. Zhu, S. He, P. He, J. Liu, and M. R. Lyu. Loghub : A Large Collection of System Log Datasets for AI-driven Log Analytics, Sept. 2023.